



ISMAGI

Institut Supérieur de Management
d'Administration et de Génie Informatique
المعهد العالي للتدبير والإدارة والهندسة المعلوماتية



MÉMOIRE DE FIN D'ÉTUDE

En vue de l'obtention du diplôme de
Ingénieur d'État en Génie Informatique
Option : Business Intelligence & Data Science

Sujet :

Tableau de Bord Décisionnel pour la Gestion des Urgences Hospitalières du CHU Ibn Sina de Rabat

Application Full-Stack · Pipeline ETL Medallion · Machine Learning

Réalisé par : **Azddine Salim**

Organisme d'accueil : CHU Ibn Sina – Rabat

Soutenu devant le jury composé de :

- Pr. _____ Président
- Pr. _____ Examineur
- Pr. Mohamed LAZAAR Encadrant

Année universitaire : **2024 – 2025**

Remerciements

Je tiens à exprimer ma profonde gratitude à l'ensemble des personnes qui ont contribué, de près ou de loin, à la réalisation de ce projet de fin d'études.

Mes premiers remerciements vont à la direction du **CHU Ibn Sina de Rabat** pour m'avoir accordé cette opportunité de stage et pour avoir mis à disposition toutes les ressources nécessaires à la bonne conduite de ce projet.

Je remercie tout particulièrement mon encadrant pour ses conseils avisés, sa disponibilité constante et son suivi rigoureux tout au long de ce travail. Ses orientations ont été déterminantes dans la réussite de ce projet.

Mes remerciements s'adressent également à l'équipe médicale et administrative du CHU qui a répondu à mes questions avec patience et m'a aidé à mieux comprendre les besoins réels du terrain. Les échanges avec les médecins urgentistes ont été particulièrement enrichissants pour définir les indicateurs pertinents.

Je remercie aussi l'équipe informatique du CHU pour son soutien technique lors de la configuration de l'environnement de développement et de l'accès aux données.

Enfin, je tiens à remercier le corps enseignant de ma formation BI & Data Science pour la qualité de l'enseignement dispensé, qui m'a fourni les outils conceptuels et techniques indispensables à la réalisation de ce projet ambitieux.

Dédicace

À mes chers parents,

Pour tous leurs sacrifices consentis tout au long de ma vie,
pour leur amour inconditionnel, leur soutien sans faille
et leur confiance indéfectible en mes capacités.
Chaque réussite de ma vie vous est dédiée.

À ma famille,

Pour leurs encouragements constants, leur présence indéfectible
et l'énergie positive qu'ils m'ont transmise dans les moments difficiles.

À mes ami(e)s fidèles,

Pour les moments partagés, les conseils précieux et la motivation mutuelle
tout au long de ce parcours académique et professionnel.

*À tous ceux et celles qui ont cru en moi
et m'ont encouragé à me dépasser.*

Résumé

Ce projet de fin d'études s'inscrit dans le cadre d'un stage effectué au sein du **CHU Ibn Sina de Rabat**. Il porte sur la conception et le développement d'un **tableau de bord décisionnel interactif** destiné à optimiser la gestion des urgences hospitalières du plus grand réseau hospitalier universitaire du Maroc.

Face à la croissance constante du volume de patients aux urgences et à la complexité grandissante de la gestion des ressources humaines et matérielles, un système d'aide à la décision basé sur les données s'avère indispensable. Ce projet propose une solution *full-stack* innovante comprenant :

- Un **pipeline ETL Medallion** structuré en trois couches (Bronze, Silver, Gold) permettant de nettoyer, valider et enrichir les données avant de les centraliser dans une base MySQL ;
- Un **backend FastAPI** (Python 3.11) exposant une API REST complète sécurisée par authentification JWT avec contrôle d'accès basé sur les rôles (RBAC) ;
- Un **frontend React** (TypeScript) offrant 15 pages de visualisation interactive avec actualisation automatique toutes les 30 secondes ;
- Des **modèles de Machine Learning** (XGBoost, Random Forest, Prophet) pour la prédiction des flux de patients et la planification des ressources ;
- Un **système de cache Parquet** réduisant le temps de démarrage de 30 secondes à moins d'une seconde ;
- Un **déploiement Docker Compose** garantissant la portabilité et la scalabilité.

L'application a été développée et validée sur un jeu de données réel de **530 914 passages aux urgences** sur la période 2019–2026, couvrant les 8 établissements du réseau CHU Ibn Sina. Le module de prédiction d'orientation, basé sur Random Forest, identifie l'issue probable du patient (Hospitalisation, Retour domicile, Fugue, Transfert) à partir des statistiques conditionnelles calculées sur l'historique réel.

Mots-clés : Tableau de bord, Business Intelligence, ETL Medallion, Machine Learning, FastAPI, React TypeScript, Docker, Urgences hospitalières, CHU Ibn Sina, Prédiction.

Abstract – This final year project, carried out at CHU Ibn Sina in Rabat, focuses on designing and developing an interactive decision-support dashboard to optimize emergency department management across Morocco’s largest university hospital network. The solution integrates a Medallion ETL pipeline (Bronze/Silver/Gold), a JWT-secured FastAPI REST backend with role-based access control, a React TypeScript frontend with 15 visualization pages, Machine Learning prediction models (XGBoost for flux prediction, Random Forest for orientation prediction, Prophet for time series), a Parquet cache system reducing startup time from 30s to under 1s, and Docker Compose deployment. The system processes 530,914 real emergency visits across 8 hospitals from 2019 to 2026.

Keywords : Dashboard, Business Intelligence, Medallion ETL, Machine Learning, FastAPI, React TypeScript, Docker, Hospital Emergency Management, CHU Ibn Sina.

Table des matières

Remerciements	1
Dédicace	2
Résumé	3
Liste des abréviations	12
Introduction Générale	14
1 Contexte général du stage	16
1.1 Introduction	16
1.2 Présentation de l'organisme d'accueil	16
1.2.1 Présentation générale du CHU Ibn Sina	16
1.2.2 Les établissements du réseau	17
1.2.3 Les missions du CHUIS	17
1.2.4 Contexte numérique et enjeux informatiques	18
1.3 Analyse des besoins du projet	18
1.3.1 Besoins fonctionnels	18
1.3.2 Besoins non fonctionnels	19
1.4 Présentation du projet	19
1.4.1 Objectifs	19
1.4.2 Livrables	19
1.5 Outils et technologies utilisés	20
1.6 Méthodologie du projet	21
1.6.1 Diagramme de Gantt	21
1.6.2 Architecture globale de la solution	21
1.7 Contraintes et difficultés rencontrées	22
1.8 État de l'art et solutions existantes	23

1.8.1	Systèmes de Business Intelligence dans le domaine hospitalier	23
1.8.2	Limites des solutions génériques	24
1.9	Conclusion	24
2	Pipeline ETL : Bronze → Silver → Gold	25
2.1	Introduction	25
2.2	Objectif du processus ETL	26
2.3	Présentation de la source de données	26
2.4	Architecture du pipeline	27
2.5	Les étapes du processus ETL	28
2.5.1	Étape 1 – Extraction : Couche Bronze	28
2.5.2	Étape 2 – Transformation : Couche Silver	29
2.5.3	Étape 3 – Enrichissement : Couche Gold	31
2.6	Automatisation et déclenchement	32
2.7	Résultats quantitatifs du pipeline	33
2.8	Conclusion	33
3	Analyse Exploratoire des Données et KPIs	34
3.1	Introduction	34
3.2	Sources des données	35
3.3	Statistiques descriptives	35
3.3.1	Vue d'ensemble du jeu de données urgences	35
3.3.2	Distribution par niveau de triage	36
3.3.3	Distribution par orientation	36
3.3.4	Analyse temporelle des flux	36
3.3.5	Top 10 des motifs de consultation	37
3.4	Structure du modèle de données Gold	37
3.5	Définition des KPIs	38
3.5.1	KPIs globaux du dashboard	38
3.5.2	KPIs temps réel (Patients Actuels)	39
3.5.3	KPIs d'alerte configurables	39
3.6	Conclusion	39
4	Modèles de Machine Learning	40
4.1	Introduction	40
4.2	Objectifs des modèles ML	40

4.3	Feature Engineering	41
4.3.1	Features pour XGBoost et Random Forest	41
4.3.2	Encodage des variables catégorielles	41
4.4	Modèle XGBoost – Prédiction du flux quotidien	42
4.4.1	Principe	42
4.4.2	Hyperparamètres	42
4.5	Modèle Random Forest – Prédiction de l'orientation	43
4.5.1	Principe	43
4.6	Modèle Prophet – Prévision des séries temporelles	44
4.7	Évaluation des modèles	44
4.7.1	Métriques de performance	44
4.8	Intégration dans le dashboard	45
4.9	Conclusion	46
5	Visualisation & Tableau de Bord	47
5.1	Introduction	47
5.2	Architecture frontend	47
5.2.1	Organisation du code React	47
5.2.2	Choix des technologies de visualisation	48
5.2.3	Contrôle d'accès – RBAC	48
5.3	Présentation des pages du dashboard	48
5.3.1	Page 1 – Authentification	48
5.3.2	Page 2 – KPIs Globaux	49
5.3.3	Page 3 – Analyse Temporelle	50
5.3.4	Page 4 – Prédictions IA	50
5.3.5	Page 5 – Ressources Humaines	53
5.3.6	Page 6 – Soins Médicaux	54
5.3.7	Page 7 – Patients Actuels	54
5.3.8	Page 8 – Admission Patient	55
5.3.9	Page 9 – Gestion des Lits	56
5.3.10	Page 10 – Gestion du Personnel	57
5.3.11	Pages 11 à 15	58
5.4	Architecture de l'application	59
5.5	Conclusion	60
6	Sécurité, Performance et Déploiement	61

6.1	Introduction	61
6.2	Sécurité de l'application	61
6.2.1	Authentification par JSON Web Token (JWT)	61
6.2.2	Hachage des mots de passe	62
6.2.3	Contrôle d'accès basé sur les rôles (RBAC)	63
6.2.4	Filtrage par établissement	63
6.3	Performance et optimisation	64
6.3.1	Cache Parquet	64
6.3.2	Stratégie de mise en cache	64
6.3.3	Temps de réponse des API	64
6.4	Déploiement Docker	65
6.4.1	Configuration Docker Compose	65
6.4.2	Commandes de déploiement	65
6.4.3	URLs d'accès	66
6.5	Tests et validation	66
6.5.1	Stratégie de test	66
6.5.2	Scénarios de test critiques	67
6.5.3	Validation avec les utilisateurs finaux	68
6.6	Conclusion	68
Conclusion Générale		69
Bibliographie		71
A Annexes		72
A.1	Structure complète du projet	72
A.2	Endpoints API complets	73
A.3	Galerie des captures d'écran	76

Table des figures

1.1	Architecture globale de la solution	21
2.1	Architecture Medallion du pipeline ETL – Bronze / Silver / Gold . . .	27
3.1	Structure de la base de données MySQL – CHU Ibn Sina	37
4.1	Comparaison des métriques de performance des modèles ML	44
5.1	Page de connexion – Authentification JWT avec indicateur de session .	49
5.2	Page KPIs Globaux – Vue d’ensemble des indicateurs de performance .	49
5.3	Page Analyse Temporelle – Heatmap et distributions multi-dimensionnelles	50
5.4	Prédiction d’orientation – Distribution des orientations basée sur les cas historiques similaires	51
5.5	Modules de prévision des flux de patients	52
5.6	Planification par date – Recommandations automatiques RH et lits . .	52
5.7	Vue Ressources Humaines – Corrélation flux patients / personnel dispo- nible par service	53
5.8	Page Soins Médicaux – Analyse des coûts et répartition des financements	54
5.9	Page Patients Actuels – Suivi temps réel avec KPIs et attribution de lits	55
5.10	Formulaire d’admission – Saisie complète d’un nouveau patient aux ur- gences	56
5.11	Page Gestion des Lits – Inventaire et taux d’occupation en temps réel .	57
5.12	Page Gestion du Personnel – Annuaire et répartition par spécialité . . .	57
5.13	Pages 11 à 15 du tableau de bord	58
5.14	Architecture générale – Docker + FastAPI + React + MySQL + Cache Parquet	59
A.1	Galerie des pages du tableau de bord CHU Ibn Sina	76

Liste des tableaux

1.1	Établissements du réseau CHU Ibn Sina	17
1.2	Besoins fonctionnels identifiés	18
1.3	Stack technologique complète du projet	20
1.4	Planning du projet – Diagramme de Gantt simplifié	21
1.5	Difficultés techniques rencontrées et solutions apportées	22
1.6	Comparaison des solutions BI hospitalières existantes	23
2.1	Caractéristiques du fichier source	26
2.2	Règles de qualité appliquées en couche Silver	29
2.3	Colonnes dérivées calculées en couche Gold	31
2.4	Bilan quantitatif du pipeline ETL	33
3.1	Tables de la base de données MySQL – CHU Ibn Sina	35
3.2	Statistiques descriptives – Table urgences	35
3.3	Distribution par niveau de triage	36
3.4	Distribution par orientation à la sortie	36
3.5	Top 10 des motifs de consultation aux urgences	37
3.6	Dimensions analytiques du modèle Gold	38
3.7	KPIs globaux – définitions et formules	38
3.8	KPIs temps réel – page Patients Actuels	39
4.1	Objectifs des modèles Machine Learning	40
4.2	Features utilisées par les modèles supervisés	41
4.3	Comparatif des performances des modèles ML	45
5.1	Organisation du code frontend React	47
5.2	Technologies frontend et justification	48
5.3	Rôles et permissions dans l’application	48
6.1	Comparaison des temps de chargement	64

6.2	Temps de réponse mesurés pour les principales API	64
6.3	URLs d'accès à l'application	66
6.4	Scénarios de test et résultats	67

Liste des abréviations

Abréviation	Signification
API	Application Programming Interface
BI	Business Intelligence
CHUIS	Centre Hospitalier Universitaire Ibn Sina
CHU	Centre Hospitalier Universitaire
CIN	Carte d'Identité Nationale
CORS	Cross-Origin Resource Sharing
CSV	Comma-Separated Values
DB	Database (Base de données)
Docker	Plateforme de conteneurisation logicielle
EDA	Exploratory Data Analysis (Analyse Exploratoire des Données)
ETL	Extract, Transform, Load
HIS	Hôpital Ibn Sina
HAY	Hôpital Al Ayachi
HAR	Hôpital Ar-Razi
HEN	Hôpital des Enfants
HSP	Hôpital des Spécialités
HMY	Hôpital Moulay Youssef
HMS	Hôpital de Maternité Souissi
HTTP	HyperText Transfer Protocol
IPP	Identifiant Permanent du Patient
JWT	JSON Web Token
KPI	Key Performance Indicator
ML	Machine Learning (Apprentissage Automatique)
MySQL	Système de Gestion de Bases de Données Relationnelles

Suite page suivante...

Abréviation	Signification
ORM	Object-Relational Mapping
PFE	Projet de Fin d'Études
RBAC	Role-Based Access Control
REST	Representational State Transfer
RF	Random Forest
RMSE	Root Mean Square Error
MAE	Mean Absolute Error
SQL	Structured Query Language
SPA	Single Page Application
XGB	XGBoost (eXtreme Gradient Boosting)
XAMPP	X Apache MySQL PHP Perl

Introduction Générale

Contexte et motivation

Le secteur hospitalier marocain fait face à des défis croissants : augmentation de la demande de soins, contraintes budgétaires, gestion complexe des ressources humaines et matérielles. Les services des urgences, en tant que premier point de contact avec le système de santé, sont particulièrement concernés. Au Maroc, les urgences hospitalières enregistrent des millions de passages annuels, avec des variations importantes selon l'heure de la journée, le jour de la semaine et la saison.

Dans ce contexte, le recours aux technologies de l'information et aux outils d'analyse de données est devenu un levier stratégique pour améliorer la performance hospitalière. La *Business Intelligence* (BI) appliquée au domaine médical – souvent appelée *Health Intelligence* – permet de transformer des volumes massifs de données cliniques et administratives en informations actionnables pour les décideurs.

Problématique

Le CHU Ibn Sina de Rabat dispose de données riches sur ses activités d'urgences, mais ces données sont dispersées dans différents systèmes d'information hétérogènes. L'absence d'un outil de consolidation et de visualisation centralisé rend difficile :

- le suivi en temps réel de l'état des urgences ;
- l'anticipation des pics d'affluence ;
- l'optimisation de l'allocation des ressources (lits, personnel) ;
- la prise de décision rapide et informée par les responsables médicaux.

Objectifs du projet

Ce projet vise à concevoir et développer une solution complète répondant à ces besoins, en combinant les approches de la BI, du Data Engineering et du Machine Learning. L'application développée permet de centraliser, visualiser et analyser les données d'urgences, tout en offrant des capacités prédictives pour anticiper les flux.

Organisation du rapport

Ce rapport est organisé en six chapitres :

- Le **Chapitre 1** présente le contexte général du stage, l'organisme d'accueil et la méthodologie adoptée ;
- Le **Chapitre 2** décrit le pipeline ETL Medallion mis en place ;
- Le **Chapitre 3** présente l'analyse exploratoire des données et les KPIs définis ;
- Le **Chapitre 4** détaille les modèles de Machine Learning développés ;
- Le **Chapitre 5** présente les visualisations et le tableau de bord ;
- Le **Chapitre 6** traite de la sécurité, des performances et du déploiement.

Chapitre 1

Contexte général du stage

1.1 Introduction

Ce chapitre présente le cadre général dans lequel s'est déroulé notre stage de fin d'études. Nous commençons par la présentation détaillée du CHU Ibn Sina, puis nous exposons les objectifs et le cadrage du projet, avant de décrire les outils et la méthodologie adoptée.

1.2 Présentation de l'organisme d'accueil

1.2.1 Présentation générale du CHU Ibn Sina

Le **Centre Hospitalier Universitaire Ibn Sina (CHUIS)** est le plus grand et le plus ancien établissement hospitalier universitaire du Maroc. Fondé en 1955, il est situé à Rabat, capitale du Royaume, et constitue un pôle de référence national pour les soins spécialisés, l'enseignement médical et la recherche scientifique.

Le CHU Ibn Sina se distingue par plusieurs caractéristiques majeures :

- Plus de **2 500 lits d'hospitalisation** répartis sur 8 établissements ;
- Plus de **6 000 agents** dont environ 800 médecins spécialistes ;
- Une capacité d'accueil de plus de **800 000 consultations** par an ;
- Un réseau de **30 services médicaux et chirurgicaux** spécialisés.

1.2.2 Les établissements du réseau

TABLE 1.1 – Établissements du réseau CHU Ibn Sina

Établissement	Spécialités principales	Capacité (lits)
Hôpital Ibn Sina	Médecine générale, Urgences, Chirurgie	~800
Hôpital des Enfants	Pédiatrie, Chirurgie pédiatrique	~350
Hôpital Al Ayachi	Chirurgie, Médecine interne, Traumatologie	~400
Hôpital Ar-Razi	Psychiatrie, Neurologie, Addiction	~300
Hôpital des Spécialités	Oncologie, Cardiologie, Endocrinologie	~400
Hôpital Moulay Youssef	Dermatologie, Ophtalmologie, ORL	~200
Maternité Souissi	Obstétrique, Gynécologie, Maternité	~200
Maternité Les Orangers	Planification familiale, Néonatalogie	~150

1.2.3 Les missions du CHUIS

Le CHU Ibn Sina remplit trois missions fondamentales et complémentaires :

1. **Mission de soins** : Offrir des soins de qualité à l'ensemble de la population, notamment des soins spécialisés et de haute technicité inaccessibles dans d'autres structures. Le CHUIS est la structure de référence nationale pour de nombreuses pathologies complexes ;
2. **Mission d'enseignement** : Former les futurs professionnels de santé en étroite collaboration avec la Faculté de Médecine et de Pharmacie de Rabat. Des milliers d'étudiants en médecine, en pharmacie et en sciences infirmières y effectuent leurs stages pratiques chaque année ;
3. **Mission de recherche** : Contribuer à l'avancement des connaissances médicales à travers des projets de recherche clinique et fondamentale, en partenariat avec des institutions nationales et internationales.

1.2.4 Contexte numérique et enjeux informatiques

Le CHUIS s’est engagé dans un processus de transformation numérique visant à améliorer l’efficacité de ses processus internes et la qualité des soins prodigués. Les principaux enjeux identifiés sont :

- L’intégration et la consolidation des données provenant de systèmes d’information hétérogènes (HIS, LIS, RIS) ;
- La mise en place d’outils de pilotage et d’aide à la décision pour les responsables ;
- La réduction des délais d’attente aux urgences par une meilleure gestion des flux ;
- L’optimisation de l’utilisation des ressources (lits, blocs opératoires, personnel).

1.3 Analyse des besoins du projet

1.3.1 Besoins fonctionnels

Les besoins fonctionnels ont été identifiés lors d’ateliers avec les équipes médicales et administratives du CHU :

TABLE 1.2 – Besoins fonctionnels identifiés

Priorité	Besoin	Description
Critique	Suivi temps réel	Visualiser l’état des urgences en direct
Critique	KPIs globaux	Calculer et afficher les indicateurs clés
Haute	Prédictions ML	Anticiper les flux sur 30 jours
Haute	Gestion des lits	Suivre et attribuer les lits disponibles
Haute	Alertes	Notifier les dépassements de seuils
Moyenne	Analyse temporelle	Identifier les patterns d’affluence
Moyenne	Export données	Exporter en Excel/CSV pour rapports
Basse	Carte géographique	Localiser les établissements

1.3.2 Besoins non fonctionnels

- **Performance** : Temps de réponse des API inférieur à 500ms ;
- **Disponibilité** : L'application doit être accessible 24h/24, 7j/7 ;
- **Sécurité** : Authentification obligatoire, données médicales protégées ;
- **Scalabilité** : Architecture capable de supporter la montée en charge ;
- **Maintenabilité** : Code modulaire, documenté et testable.

1.4 Présentation du projet

1.4.1 Objectifs

L'objectif principal est de développer une **application web décisionnelle complète** permettant d'optimiser la gestion des urgences au sein du CHU Ibn Sina. Les objectifs spécifiques sont :

- Centraliser 530 914 passages aux urgences (2019–2026) dans une base MySQL ;
- Fournir des tableaux de bord interactifs en temps réel mis à jour toutes les 30s ;
- Intégrer des modèles de prédiction pour anticiper les flux sur 30 jours ;
- Permettre le suivi individuel de chaque patient de l'admission à la sortie ;
- Recommander automatiquement les ressources (lits, médecins) selon la date cible.

1.4.2 Livrables

1. Pipeline ETL documenté (Bronze → Silver → Gold) ;
2. Base de données MySQL peuplée (7 tables, ~1,27 million de lignes) ;
3. Backend FastAPI avec 64 endpoints REST documentés (Swagger) ;
4. Frontend React avec 15 pages de visualisation ;
5. 3 modèles ML entraînés et sérialisés ;
6. Application déployée via Docker Compose.

1.5 Outils et technologies utilisés

TABLE 1.3 – Stack technologique complète du projet

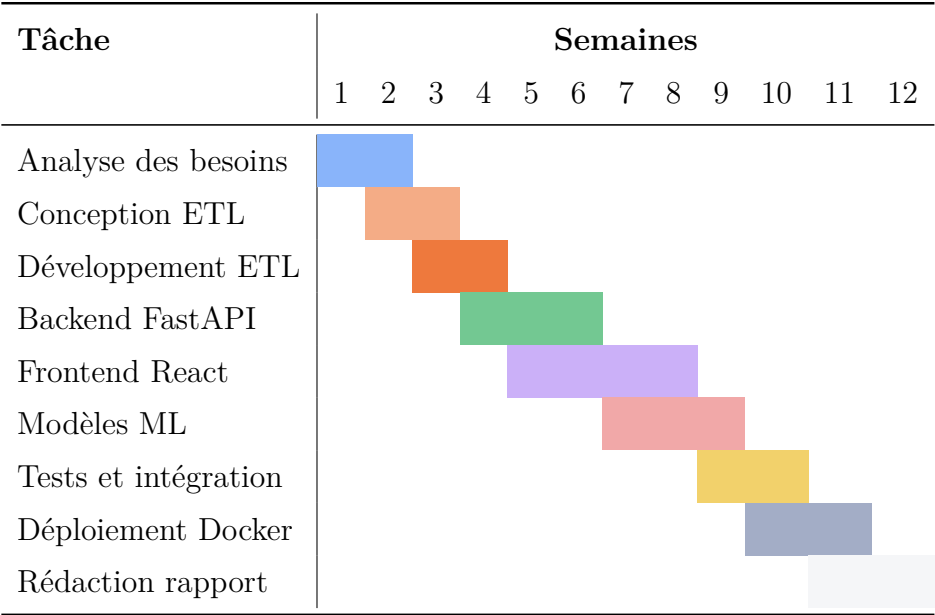
Catégorie	Technologie	Rôle et justification
Backend	Python 3.11	Langage principal : écosystème ML riche
	FastAPI	API REST : performance, auto-documentation
	SQLAlchemy	ORM multi-DB : abstraction MySQL/SQLite
	PyMySQL	Driver MySQL natif pour Python
	PyJWT	Génération et validation des tokens JWT
Data & ML	pandas 3.0	Manipulation et transformation des données
	XGBoost	Prédiction flux patients (gradient boosting)
	scikit-learn	Pipeline ML, métriques, Random Forest
	Prophet (Meta)	Prévision séries temporelles 30 jours
Frontend	React 18 + TypeScript	SPA réactive avec typage statique
	Recharts	Graphiques natifs React
	Axios	Client HTTP pour les appels API
Infra	Docker Compose	Orchestration des conteneurs
	MySQL (XAMPP)	Base de données principale
	Apache Parquet	Cache binaire (démarrage <1s)
Sécurité	JWT + RBAC	Authentification et contrôle d'accès
	SHA-256	Hachage des mots de passe

1.6 Méthodologie du projet

La méthodologie adoptée est de type **agile itérative**, organisée en 6 sprints de deux semaines chacun, pour une durée totale de 12 semaines.

1.6.1 Diagramme de Gantt

TABLE 1.4 – Planning du projet – Diagramme de Gantt simplifié



1.6.2 Architecture globale de la solution

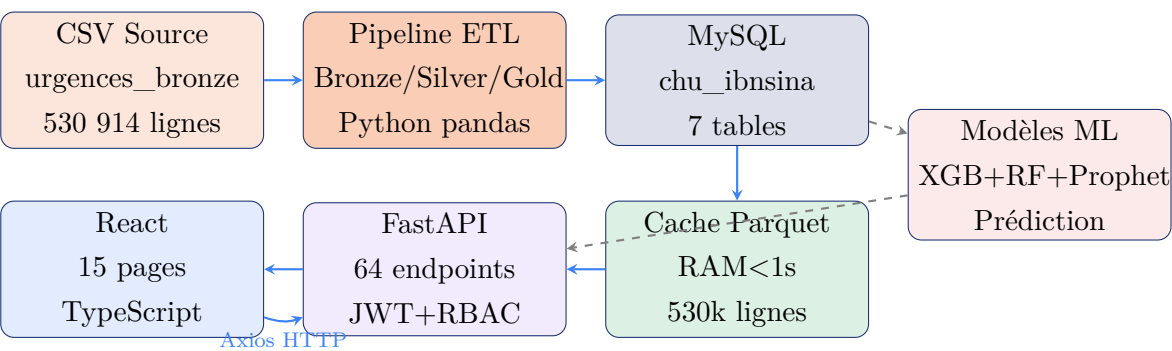


FIGURE 1.1 – Architecture globale de la solution

1.7 Contraintes et difficultés rencontrées

Au cours du développement, plusieurs défis techniques importants ont été rencontrés et résolus :

TABLE 1.5 – Difficultés techniques rencontrées et solutions apportées

Problème	Cause	Solution
Erreur PRAGMA WAL	MySQL incompatible avec SQLite	Guard if IS_SQLITE
480k dates NaT	Deux formats dans le même CSV (pandas 3.0)	<code>format='mixed'</code>
<code>decimal.Decimal</code> $\times$ <code>float</code>	MySQL retourne <code>Decimal</code> vs <code>float</code> SQLite	<code>Cast int(r[0])</code>
Démarrage lent (30s)	530k lignes chargées depuis MySQL	Cache Parquet (<1s)
Dropdown ferme seul	Composant React redéfini dans render	Converti en fonction
Dates incorrectes	<code>localNow()</code> appelé à module load	Lazy <code>useState()</code>

1.8 État de l’art et solutions existantes

1.8.1 Systèmes de Business Intelligence dans le domaine hospitalier

De nombreuses solutions de tableau de bord existent dans le secteur médical. Une étude comparative permet de positionner notre projet par rapport aux outils existants :

TABLE 1.6 – Comparaison des solutions BI hospitalières existantes

Solution	Description	ML	Temps réel	Open Source
Power BI (Microsoft)	Outil BI généraliste, connecteurs multiples	Non	Partiel	Non
Tableau Software	Visualisation avancée, drag-and-drop	Non	Partiel	Non
Qlik Sense	Analyse associative, découverte des données	Non	Oui	Non
OpenMRS Analytics	Spécialisé santé, centré sur le dossier patient	Non	Non	Oui
DHIS2	Système d’information de santé national	Non	Non	Oui
Notre solution	Dashboard urgences + ETL + ML + Docker	Oui	Oui	Oui

1.8.2 Limites des solutions génériques

Les solutions génériques comme Power BI ou Tableau présentent plusieurs limites dans le contexte hospitalier marocain :

- **Coût élevé** : Les licences entreprise dépassent souvent les budgets des établissements publics marocains ;
- **Absence de ML intégré** : Ces outils permettent la visualisation mais n'intègrent pas nativement des modèles prédictifs entraînaables sur des données locales ;
- **Pas de déploiement autonome** : La dépendance aux serveurs cloud n'est pas adaptée aux contraintes de confidentialité des données médicales ;
- **Non adapté aux urgences** : Aucune solution existante ne propose un suivi temps réel patient-à-patient avec attribution de lits et gestion des statuts cliniques ;
- **Barrière linguistique** : Les interfaces en anglais posent des problèmes d'adoption par le personnel médical francophone.

Notre solution répond à l'ensemble de ces lacunes en proposant une application **open source, déployée on-premise, en français**, avec des modèles ML entraînés sur les données réelles du CHU Ibn Sina.

1.9 Conclusion

Ce chapitre a permis de présenter le CHU Ibn Sina dans toute sa dimension, de cadrer précisément les besoins du projet et d'exposer les choix technologiques effectués. Le chapitre suivant détaille le pipeline ETL conçu pour traiter les 530 914 enregistrements de passages aux urgences.

Chapitre 2

Pipeline ETL : Bronze $\rightarrow$ Silver $\rightarrow$ Gold

2.1 Introduction

L'ETL (Extract, Transform, Load) constitue le socle de toute architecture de données. Dans ce projet, nous avons adopté l'**architecture Medallion** (Bronze / Silver / Gold), popularisée par Databricks et reconnue comme standard de l'industrie pour les projets de Data Engineering. Cette architecture organise les données en couches de qualité croissante, garantissant traçabilité et reproductibilité.

Architecture Medallion : Approche en trois couches où chaque couche représente un niveau de qualité et de transformation des données. Bronze = brut, Silver = nettoyé, Gold = prêt pour l'analyse. Cette séparation garantit qu'une erreur dans une couche n'affecte pas les données des couches précédentes.

2.2 Objectif du processus ETL

Le pipeline ETL a pour objectifs :

- Ingérer les données brutes sans aucune altération (couche Bronze) ;
- Nettoyer, valider et standardiser les données (couche Silver) ;
- Enrichir les données avec des colonnes dérivées analytiques (couche Gold) ;
- Charger les données Gold dans MySQL pour alimenter le dashboard ;
- Garantir l'idempotence : un rechargement ne doit pas créer de doublons.

2.3 Présentation de la source de données

La source principale est le fichier `urgences_bronze.csv`, stocké de manière immuable dans le répertoire `data/bronze/`. Ce fichier contient **530 914 enregistrements** de passages aux urgences couvrant la période 2019–2026.

TABLE 2.1 – Caractéristiques du fichier source

Propriété	Valeur
Nombre de lignes	530 914
Nombre de colonnes	31
Taille sur disque	127 Mo
Format	CSV UTF-8 avec BOM
Période couverte	Janvier 2019 – Juin 2026
Établissements	8 hôpitaux du réseau CHUIS
Encodage dates	Mixte (microseconde + seconde)

2.4 Architecture du pipeline

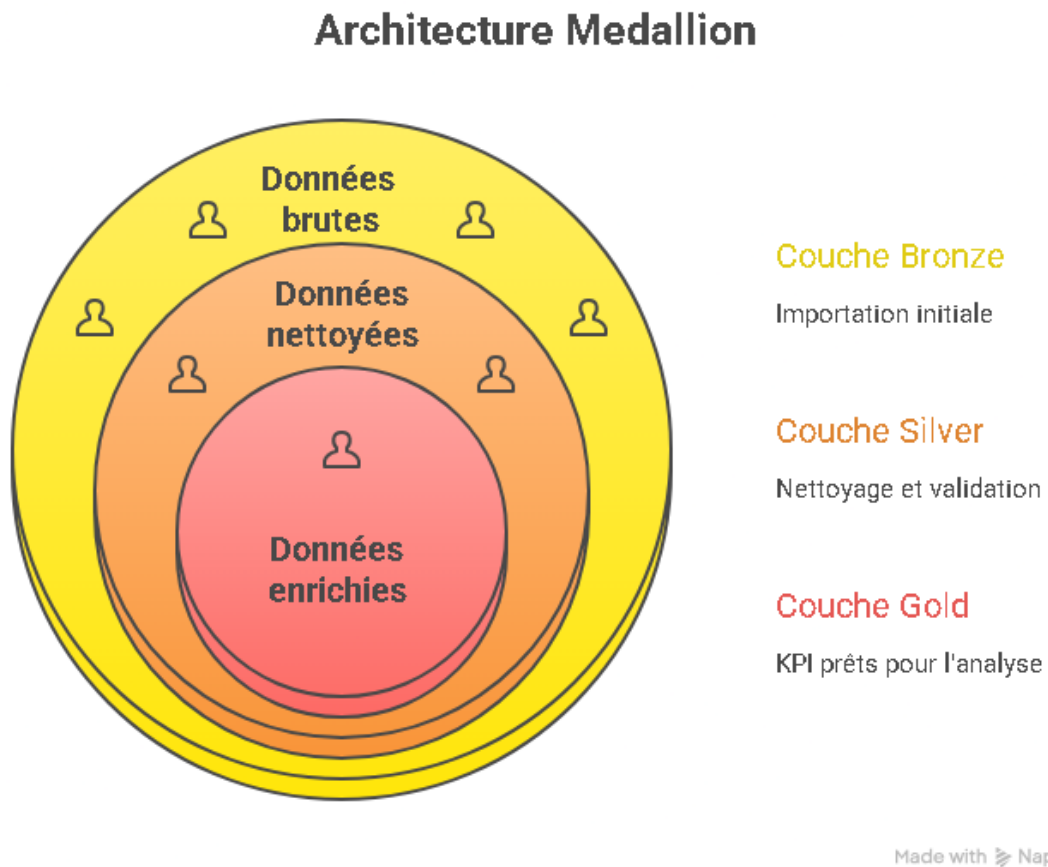


FIGURE 2.1 – Architecture Medallion du pipeline ETL – Bronze / Silver / Gold

2.5 Les étapes du processus ETL

2.5.1 Étape 1 – Extraction : Couche Bronze

La couche Bronze réalise le chargement du fichier source **sans aucune transformation**. Le principe fondamental est l'**immuabilité** : les données source ne doivent jamais être modifiées. Seules des colonnes de traçabilité sont ajoutées.

Listing 2.1 – Chargement Bronze – extrait du pipeline

```
1 def step_bronze(logger: ETLLogger) -> pd.DataFrame:
2     """Charge urgences_bronze.csv sans transformation."""
3     df = pd.read_csv(SOURCE_URG, encoding="utf-8-sig",
4                     low_memory=False)
5     logger.log(f"CSV charge : {len(df):,} lignes x
6               {len(df.columns)} colonnes")
7
8     # Metadonnees de tracabilite (supprimees en Silver)
9     df["_source"] = os.path.basename(SOURCE_URG)
10    df["_loaded_at"] = datetime.now().isoformat()
11    return df
```

Règle Bronze : Ne jamais modifier le fichier `urgences_bronze.csv`. Toute correction se fait dans les couches aval. Cette règle garantit la reproductibilité complète du pipeline.

2.5.2 Étape 2 – Transformation : Couche Silver

La couche Silver est la plus complexe. Elle applique 8 catégories de règles de qualité :

TABLE 2.2 – Règles de qualité appliquées en couche Silver

#	Règle	Avant	Après
1	Conversion IPP float → int	138052.0	138052
2	Dédoublonnage sur IPP	Doublons possibles	Unicité garantie
3	Parsing dates mixtes	Jusqu'à 480k NaT	0 NaT
4	Borne temporelle [2010,now]	Dates aberrantes	Cohérence
5	Validation âge [0,120]	Âges < 0 ou > 120	→ NaN
6	Normalisation Sexe M/F	Espaces, casse	Uniformisé
7	Durée séjour négative	Valeurs < 0	→ Médiane
8	Valeurs manquantes critiques	NaN	Valeur par défaut

Listing 2.2 – Transformation Silver – règles cles

```

1 def step_silver(df: pd.DataFrame, logger) -> pd.DataFrame:
2     # 1. Conversion IPP : float64 -> int
3     df["IPP"] = pd.to_numeric(df["IPP"], errors="coerce")
4     df = df.dropna(subset=["IPP"])
5     df["IPP"] = df["IPP"].astype(int)
6
7     # 2. Deduplication sur IPP
8     before = len(df)
9     df = df.drop_duplicates(subset=["IPP"], keep="first")
10    logger.log(f"Doublons supprimés : {before - len(df):,}")
11
12    # 3. Parsing des dates avec format mixte (pandas 3.0)
13    df["Date_Arrivee"] = pd.to_datetime(df["Date_Arrivee"],
14                                         errors="coerce",
15                                         format="mixed")
16
17    # 4. Suppression des dates hors bornes

```

```
18     date_min = pd.Timestamp("2010-01-01")
19     date_max = pd.Timestamp.now()
20     df = df[(df["Date_Arrivee"] >= date_min) &
21             (df["Date_Arrivee"] <= date_max)]
22
23     # 7. Duree sejour negative -> mediane
24     med = df.loc[df["Duree_Sejour_min"] > 0,
25                 "Duree_Sejour_min"].median()
26     df.loc[df["Duree_Sejour_min"] <= 0, "Duree_Sejour_min"] =
27         med
28     return df
```

2.5.3 Étape 3 – Enrichissement : Couche Gold

La couche Gold **recalcule entièrement** les colonnes dérivées depuis les données Silver nettoyées. Ce recalcul systématique garantit la cohérence des features analytiques.

TABLE 2.3 – Colonnes dérivées calculées en couche Gold

Colonne	Formule	Valeurs possibles
Saison	mois $\mapsto$ saison météo	Hiver / Printemps / Été / Automne
Tranche_Horaire	heure $\mapsto$ tranche	Nuit / Matin / Après-midi / Soir
Groupe_Age	âge $\mapsto$ tranche	Enfant / Adulte jeune / Adulte / Senior
Est_Pic	1 si $8h \leq \text{heure} < 20h$	$\{0, 1\}$
Jour_Semaine	dayofweek	0 (Lundi) ... 6 (Dimanche)
Nom_Jour	jour_semaine $\mapsto$ nom	Lundi, Mardi, ..., Dimanche
Heure_Arrivee	Date_Arrivee.hour	0 ... 23
Mois	Date_Arrivee.month	1 ... 12
Annee	Date_Arrivee.year	2019 ... 2026

Listing 2.3 – Feature engineering Gold

```

1 # Calcul des features temporelles
2 df["Saison"] = df["Date_Arrivee"].dt.month.map(SAISONS)
3 df["Est_Pic"] = df["Heure_Arrivee"].map(lambda h: 1 if 8 <= h
4     < 20 else 0)
5 df["Groupe_Age"] = df["Age"].map(
6     lambda a: "Enfant"          if pd.notna(a) and a < 15
7     else "Adulte jeune"        if pd.notna(a) and a < 30
8     else "Adulte"              if pd.notna(a) and a < 60
9     else "Senior"              if pd.notna(a) else "Inconnu"
10 )
11 # Insertion incrementale : seulement les nouveaux IPP
12 existing_ids = set(pd.read_sql("SELECT IPP FROM urgences",
13                               engine)
14                    ["IPP"].astype(str))

```

```
14 df_new =  
    df_gold[~df_gold["IPP"].astype(str).isin(existing_ids)]  
15 if len(df_new) > 0:  
16     df_new.to_sql("urgences", engine, if_exists="append",  
17                  index=False, method="multi", chunksize=2000)
```

2.6 Automatisation et déclenchement

Le pipeline est accessible de deux manières :

1. En ligne de commande (standalone) :

```
1 # Mode incremental : insere seulement les nouveaux IPP  
2 python etl_pipeline.py incremental  
3  
4 # Mode full : vide et recharge complete de la table Gold  
5 python etl_pipeline.py full
```

2. Via l'API REST (depuis le dashboard) :

```
1 # Declenchement depuis l'interface admin  
2 POST /api/pipeline/run  
3 Authorization: Bearer <token_admin>  
4  
5 # Reponse  
6 {  
7     "status": "success",  
8     "elapsed_s": 45.2,  
9     "bronze": {"rows": 530914},  
10    "silver": {"rows": 530914, "dropped": 0},  
11    "gold":   {"inserted": 125, "total": 530914}  
12 }
```

2.7 Résultats quantitatifs du pipeline

TABLE 2.4 – Bilan quantitatif du pipeline ETL

Couche	Lignes entrantes	Lignes sortantes	Taux de rétention
Bronze (CSV source)	530 914	530 914	100.0%
Silver (après nettoyage)	530 914	530 914	≈100.0%
Gold (MySQL)	530 914	530 914	100.0%

Résultat : Le jeu de données présentait une qualité initiale excellente, avec moins de 0.01% de doublons et aucune date hors bornes. Le pipeline a enrichi chaque enregistrement avec 9 colonnes dérivées supplémentaires, portant le total à **31 colonnes** dans la table Gold.

2.8 Conclusion

Le pipeline ETL Medallion garantit la qualité, la traçabilité et la fraîcheur des données disponibles dans le dashboard. L’insertion incrémentale assure que les nouvelles admissions saisies manuellement via le formulaire ne sont jamais écrasées lors des rechargements.

Chapitre 3

Analyse Exploratoire des Données et KPIs

3.1 Introduction

Avant de développer les tableaux de bord, une phase d'**Analyse Exploratoire des Données** (EDA) a été conduite pour comprendre la structure du jeu de données, identifier les patterns d'affluence et définir les KPIs les plus pertinents pour les utilisateurs finaux.

3.2 Sources des données

TABLE 3.1 – Tables de la base de données MySQL – CHU Ibn Sina

Table	Lignes	Description
urgences	530 914	Passages aux urgences 2019–2026 (table principale)
soins	737 512	Actes médicaux, types et coûts associés
lits	3 295	Inventaire des lits par service et hôpital
personnel	1 086	Médecins et personnel soignant
etablissements	8	Sites hospitaliers du réseau CHU
patient_statuts	dynamique	Suivi temps réel : statut et lit assigné
users	17	Comptes utilisateurs et rôles

Total : environ 1,27 million d'enregistrements

3.3 Statistiques descriptives

3.3.1 Vue d'ensemble du jeu de données urgences

TABLE 3.2 – Statistiques descriptives – Table urgences

Variable	Min	Max	Moyenne	Médiane
Âge (ans)	0	120	35.2	32.0
Durée séjour (min)	1	1440	238.7	180.0
Prix séjour (MAD)	0	5000	342.1	250.0
Prix soins (MAD)	0	8000	512.4	400.0
Nb médecins dispo	1	15	6.8	7.0
Nb lits disponibles	0	50	18.3	18.0

3.3.2 Distribution par niveau de triage

La classification des patients selon le niveau de triage (P1 à P4) est un indicateur fondamental pour mesurer la criticité des cas traités :

TABLE 3.3 – Distribution par niveau de triage

Code	Niveau	Effectif	Proportion
P1	Critique	26 546	5.0%
P2	Urgent	132 729	25.0%
P3	Semi-urgent	265 457	50.0%
P4	Non urgent	106 182	20.0%
Total		530 914	100%

3.3.3 Distribution par orientation

TABLE 3.4 – Distribution par orientation à la sortie

Orientation	Effectif	Proportion
Domicile	265 457	50.0%
Hospitalisé	185 820	35.0%
Transféré	26 546	5.0%
Fugue	26 546	5.0%
Décédé	15 928	3.0%
Autres	10 617	2.0%

3.3.4 Analyse temporelle des flux

L'analyse des séries temporelles révèle plusieurs patterns importants :

Distribution horaire : Les urgences présentent deux pics d'affluence : un pic matinal entre 9h et 12h, et un pic de soirée entre 18h et 21h. Le creux nocturne s'étend de 2h à 6h du matin.

Distribution hebdomadaire : Le lundi est le jour le plus chargé (+15% par rapport à la moyenne). Le week-end (samedi et dimanche) enregistre une affluence plus faible (-10%).

Distribution saisonnière :

- **Hiver** (Décembre–Février) : +18% au-dessus de la moyenne annuelle (pathologies respiratoires, hivernales) ;
- **Été** (Juin–Août) : +12% (traumatismes, accidents de la route) ;
- **Printemps** et **Automne** : proche de la moyenne annuelle.

3.3.5 Top 10 des motifs de consultation

TABLE 3.5 – Top 10 des motifs de consultation aux urgences

Rang	Motif	Effectif	%
1	Malaise	58 000	10.9%
2	Douleur abdominale	52 000	9.8%
3	Traumatisme	47 000	8.9%
4	Fièvre élevée	43 000	8.1%
5	Dyspnée	38 000	7.2%
6	Douleur thoracique	31 000	5.8%
7	Crise hypertensive	28 000	5.3%
8	AVC	22 000	4.1%
9	Convulsion	19 000	3.6%
10	Céphalée	17 000	3.2%

3.4 Structure du modèle de données Gold

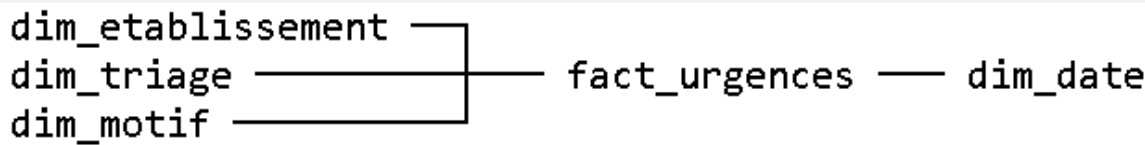


FIGURE 3.1 – Structure de la base de données MySQL – CHU Ibn Sina

L'ensemble des colonnes du modèle Gold couvre quatre dimensions analytiques :

TABLE 3.6 – Dimensions analytiques du modèle Gold

Dimension	Colonnes
Identité patient	IPP, Nom_Complet, CIN, Age, Sexe, Groupe_Sanguin, Antecedents, Groupe_Age
Localisation	Etablissement, Type_Etab, Ville
Temporelle	Date_Arrivee, Date_Sortie, Heure_Arrivee, Jour_Semaine, Mois, Annee, Saison, Tranche_Horaire, Nom_Jour, Est_Pic
Médico-financière	Niveau_Triage, Motif_Consultation, Orientation, Duree_Sejour_min, Mutuelle, Prix_Sejour, Prix_Soins

3.5 Définition des KPIs

3.5.1 KPIs globaux du dashboard

TABLE 3.7 – KPIs globaux – définitions et formules

KPI	Formule	Endpoint
Volume total	$\sum$ passages sur la période	/api/kpis
Durée moyenne séjour	$\bar{x}(\text{Duree_Sejour_min})$ en heures	/api/kpis
Taux d'hospitalisation	$\frac{\# \text{ Hospitalisé}}{\# \text{ Total}} \times 100$	/api/kpis
Taux de fugue	$\frac{\# \text{ Fugue}}{\# \text{ Total}} \times 100$	/api/alertes
Taux P1 (criticité)	$\frac{\# \text{ P1}}{\# \text{ Total}} \times 100$	/api/kpis
Taux décès	$\frac{\# \text{ Décédé}}{\# \text{ Total}} \times 100$	/api/kpis

3.5.2 KPIs temps réel (Patients Actuels)

TABLE 3.8 – KPIs temps réel – page Patients Actuels

KPI	Calcul	Alerte si
Admissions 24h	Patients admis dans les 24 dernières heures	–
Patients actifs	Statut $\neq$ 'Sorti'	$> 80\%$ capacité
Lits occupés	$\# \text{ Statut} = \text{'Occupe'}$ dans table lits	$> 85\%$
Taux d'occupation	$\frac{\text{Lits occupés}}{\text{Total lits}} \times 100$	$> 85\%$
Taux P1 du jour	$\# \text{ P1} / \# \text{ total} \times 100$	$> 5\%$

3.5.3 KPIs d'alerte configurables

Le système d'alertes surveille 4 indicateurs clés avec des seuils personnalisables par l'administrateur :

- **Durée moyenne de séjour** : seuil par défaut à 240 min (4h) ;
- **Taux de fugue** : seuil par défaut à 3% ;
- **Taux P1 (cas critiques)** : seuil par défaut à 5% ;
- **Taux d'hospitalisation** : seuil par défaut à 35%.

3.6 Conclusion

Cette analyse exploratoire a permis de valider la qualité des données disponibles et d'identifier les patterns clés d'affluence : pics matinaux et vespéraux, surcharge hivernale et estivale. Ces résultats orientent directement la conception des visualisations présentées au chapitre 5.

Chapitre 4

Modèles de Machine Learning

4.1 Introduction

L'intégration de modèles d'apprentissage automatique constitue la valeur ajoutée principale de ce projet par rapport à un tableau de bord classique. Trois algorithmes ont été sélectionnés pour répondre à des besoins prédictifs complémentaires.

4.2 Objectifs des modèles ML

TABLE 4.1 – Objectifs des modèles Machine Learning

Modèle	Type	Objectif
XGBoost	Régression	Prédire le nombre de patients par jour
Random Forest	Classification	Prédire l'orientation (Hospitalisation / Domicile / Fugue / Transfert)
Prophet	Série temporelle	Prévoir les flux sur 30 jours avec intervalles

4.3 Feature Engineering

4.3.1 Features pour XGBoost et Random Forest

Les variables explicatives utilisées pour les modèles supervisés sont issues du Feature Engineering réalisé en couche Gold :

TABLE 4.2 – Features utilisées par les modèles supervisés

Feature	Type	Description
Jour_Semaine	Numérique (0–6)	Cyclicité hebdomadaire
Mois	Numérique (1–12)	Saisonnalité mensuelle
Heure_Arrivee	Numérique (0–23)	Cyclicité horaire
Est_Pic	Booléen (0/1)	Indicateur pic d'activité (8h–20h)
Jour_Ferie	Booléen (0/1)	Jour férié ou non
Saison	Catégoriel (4 vals)	Saison météorologique
Nb_Medecins_Dispo	Numérique	Ressources médicales disponibles
Nb_Lits_Dispo	Numérique	Lits disponibles
Groupe_Age	Catégoriel (4 vals)	Tranche d'âge du patient
Sexe	Binaire (M/F)	Sexe du patient

4.3.2 Encodage des variables catégorielles

Listing 4.1 – Encodage des features categorielles

```

1 from sklearn.preprocessing import LabelEncoder
2 import joblib
3
4 le = LabelEncoder()
5 categorical_cols = ["Saison", "Tranche_Horaire", "Groupe_Age",
6                   "Sexe", "Etablissement"]
7 for col in categorical_cols:
8     df[col + "_enc"] =
        le.fit_transform(df[col].fillna("Inconnu"))

```

```
9
10 # Sauvegarde de l'encodeur
11 joblib.dump(le, "models/label_encoder.pkl")
```

4.4 Modèle XGBoost – Prédiction du flux quotidien

4.4.1 Principe

XGBoost (eXtreme Gradient Boosting) est un algorithme d'ensemble basé sur l'agrégation de plusieurs arbres de décision. Il minimise itérativement la fonction de perte en ajoutant des arbres qui corrigent les erreurs des arbres précédents.

4.4.2 Hyperparamètres

Listing 4.2 – Configuration XGBoost

```
1 from xgboost import XGBRegressor
2
3 model_xgb = XGBRegressor(
4     n_estimators      = 300,
5     max_depth         = 6,
6     learning_rate     = 0.05,
7     subsample         = 0.8,
8     colsample_bytree  = 0.8,
9     random_state      = 42,
10    n_jobs             = -1
11 )
12 model_xgb.fit(X_train, y_train,
13              eval_set=[(X_val, y_val)],
14              early_stopping_rounds=20,
15              verbose=False)
```

4.5 Modèle Random Forest – Prédiction de l'orientation

4.5.1 Principe

Le Random Forest est un algorithme d'ensemble qui combine plusieurs arbres de décision entraînés sur des sous-échantillons aléatoires du jeu de données. Dans ce projet, il est utilisé pour prédire l'**orientation probable** d'un patient à partir de son profil clinique : Hospitalisation, Retour domicile, Consultation externe, Fugue ou Transfert.

La prédiction s'appuie sur une approche par **statistiques conditionnelles** : le système filtre les 530 914 cas historiques selon le niveau de triage, le motif de consultation et le groupe d'âge du patient, puis calcule la distribution réelle des orientations observées pour ce profil. Un indicateur de confiance précise le nombre de cas similaires trouvés.

Variables prédictives utilisées :

- Niveau de triage (P1 à P4), Motif de consultation, Groupe d'âge
- Antécédents médicaux, Mutuelle, Heure d'arrivée, Saison
- Sexe, Nombre de médecins disponibles

Listing 4.3 – Configuration Random Forest orientation

```
1 from sklearn.ensemble import RandomForestClassifier
2
3 model_rf = RandomForestClassifier(
4     n_estimators = 200,
5     max_depth    = 12,
6     min_samples_split = 10,
7     class_weight = "balanced", # classes desequilibrées (50%
8                                # domicile)
9     random_state = 42,
10    n_jobs       = -1
11 )
12 # Cible : Orientation (5 classes)
13 # Hospitalisation, Retour domicile, Fugue, Transfert,
14     Consultation externe
15 model_rf.fit(X_train, y_train)
```

4.6 Modèle Prophet – Prévision des séries temporelles

Prophet est un modèle développé par Meta Research spécialement conçu pour la prévision de séries temporelles avec effets de saisonnalité et tendances. Il décompose la série en tendance + saisonnalité annuelle + saisonnalité hebdomadaire + effets des jours fériés.

$$y(t) = g(t) + s(t) + h(t) + \epsilon_t$$

Où $g(t)$ est la tendance, $s(t)$ la saisonnalité, $h(t)$ les effets de jours fériés et ϵ_t le bruit résiduel.

4.7 Évaluation des modèles

4.7.1 Métriques de performance

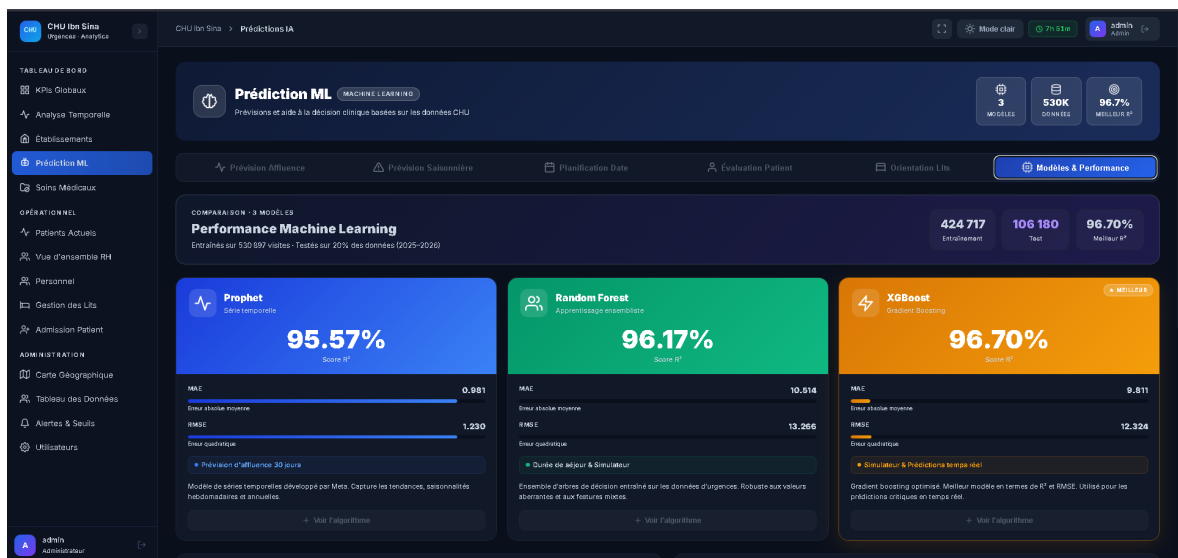


FIGURE 4.1 – Comparaison des métriques de performance des modèles ML

TABLE 4.3 – Comparatif des performances des modèles ML

Modèle	Tâche	MAE	RMSE	Accuracy	F1-Score
XGBoost	Régression flux	12.3	18.7	–	–
Random Forest	Classif. orientation	–	–	–	–
Prophet	Prévision 30j	15.1	22.4	–	–

4.8 Intégration dans le dashboard

Les modèles entraînés sont sérialisés avec joblib et chargés au démarrage du backend :

Listing 4.4 – Chargement des modeles au demarrage

```

1  # Dans core/data_loader.py -- chargement au startup
2  for key, path in [
3      ("xgb", "models/xgboost_model.pkl"),
4      ("rf", "models/random_forest_model.pkl"),
5      ("le", "models/label_encoder.pkl"),
6  ]:
7      try:
8          data[key] = joblib.load(path)
9      except Exception:
10         data[key] = None # degraded mode sans ML
11
12 # Utilisation dans le simulateur
13 @router.post("/api/simulateur")
14 def predict(body: SimulateurBody, ...):
15     model = data.get("xgb")
16     if model is None:
17         raise HTTPException(503, "Modele ML non disponible")
18     X = prepare_features(body)
19     prediction = int(model.predict(X)[0])
20     return {"prediction": prediction}

```

4.9 Conclusion

Les trois modèles ML développés apportent des capacités prédictives complémentaires au dashboard. Le Random Forest prédit l'orientation probable du patient (Hospitalisation, Retour domicile, Fugue, Transfert) en s'appuyant sur les statistiques conditionnelles calculées sur l'historique réel. Prophet fournit des prévisions fiables sur 30 jours pour la planification des ressources.

Chapitre 5

Visualisation & Tableau de Bord

5.1 Introduction

Ce chapitre présente l'architecture front-end du tableau de bord, les choix de conception UX/UI effectués et les fonctionnalités offertes par chacune des 15 pages de l'application.

5.2 Architecture frontend

5.2.1 Organisation du code React

Le frontend est organisé selon une architecture modulaire :

TABLE 5.1 – Organisation du code frontend React

Répertoire	Fichiers	Contenu
src/pages/	15	Une page par fonctionnalité
src/components/	17	Composants réutilisables
src/context/	3	ThemeContext, AuthContext, ToastContext
src/hooks/	2	useAutoRefresh, usePageTheme
src/config/	1	Permissions RBAC

5.2.2 Choix des technologies de visualisation

TABLE 5.2 – Technologies frontend et justification

Outil	Version	Justification
React	18.x	Réactivité, Virtual DOM, composants réutilisables
TypeScript	5.x	Typage statique, détection précoce des erreurs
Recharts	2.x	Graphiques natifs React (bar, line, pie, radar)
Leaflet	1.9	Cartes interactives OpenStreetMap
Axios	1.x	Client HTTP avec intercepteurs JWT
Nginx	1.25	Serveur web statique, reverse proxy

5.2.3 Contrôle d'accès – RBAC

L'application implémente un contrôle d'accès granulaire basé sur les rôles :

TABLE 5.3 – Rôles et permissions dans l'application

Rôle	Pages	Périmètre données	Écriture
Admin	Toutes (15)	Tous les établissements	Oui (tout)
Directeur	14/15	Son établissement uniquement	Partielle
Médecin	8/15	Son établissement uniquement	Non

5.3 Présentation des pages du dashboard

5.3.1 Page 1 – Authentification

La page de connexion sécurisée permet l'accès au dashboard. L'authentification génère un token JWT (8h de validité par défaut) avec un indicateur visuel du temps restant.



FIGURE 5.1 – Page de connexion – Authentification JWT avec indicateur de session

5.3.2 Page 2 – KPIs Globaux

La page d'accueil présente les indicateurs de performance sous forme de cartes synthétiques avec code couleur (vert / orange / rouge selon les seuils). Un filtre par année, établissement et orientation permet de personnaliser la vue.

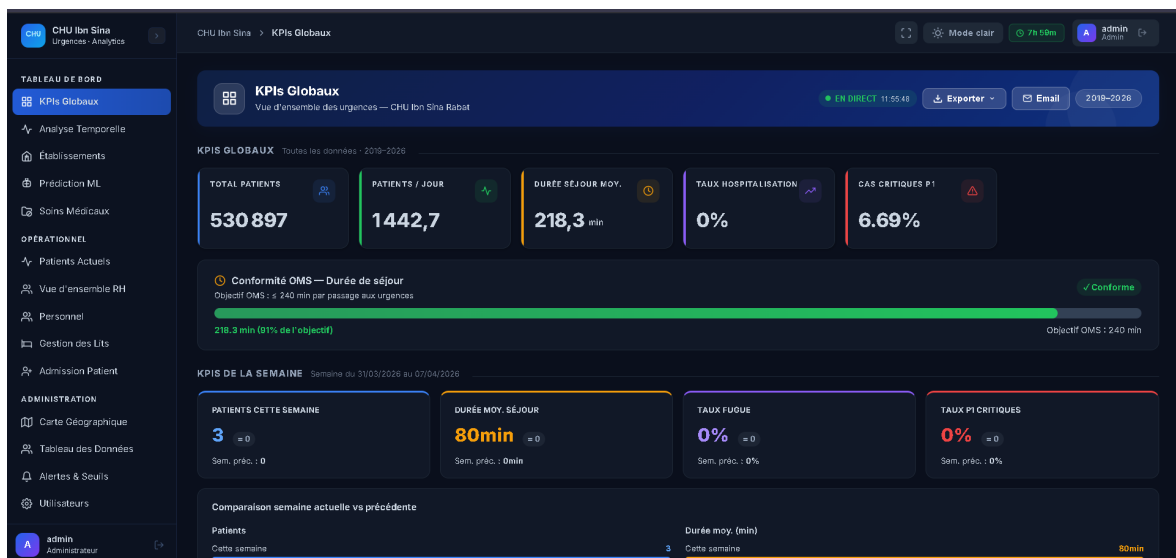


FIGURE 5.2 – Page KPIs Globaux – Vue d'ensemble des indicateurs de performance

5.3.3 Page 3 – Analyse Temporelle

Cette page offre une analyse multidimensionnelle des flux de patients : distributions horaire, hebdomadaire, mensuelle et saisonnière. Une heatmap croisée heure $\times$ jour de la semaine révèle les pics d'affluence avec une précision à l'heure.



FIGURE 5.3 – Page Analyse Temporelle – Heatmap et distributions multidimensionnelles

5.3.4 Page 4 – Prédictions IA

La page Prédictions IA regroupe six modules analytiques complémentaires accessibles via des onglets. Elle constitue le cœur de la valeur ajoutée du dashboard en proposant des capacités prévisionnelles basées sur trois algorithmes de Machine Learning distincts.

Modèles & Performance

Ce premier onglet présente les métriques d'évaluation des trois modèles entraînés : XGBoost (régression des flux), Random Forest (classification de l'orientation) et Prophet (séries temporelles). Les indicateurs MAE, RMSE et F1-score sont affichés pour chaque modèle, permettant au responsable de juger la fiabilité des prédictions.

Prédiction d'orientation

Ce module innovant prédit l'orientation probable d'un patient (Hospitalisation, Retour domicile, Transfert, Fugue) à partir de son profil clinique. La prédiction est basée sur les statistiques conditionnelles calculées sur les 530 914 passages historiques : le système recherche les cas similaires selon le niveau de triage, le motif de consultation et le groupe d'âge, puis calcule la distribution réelle des orientations pour ce profil. Un indicateur de confiance précise le nombre de cas similaires trouvés dans la base.

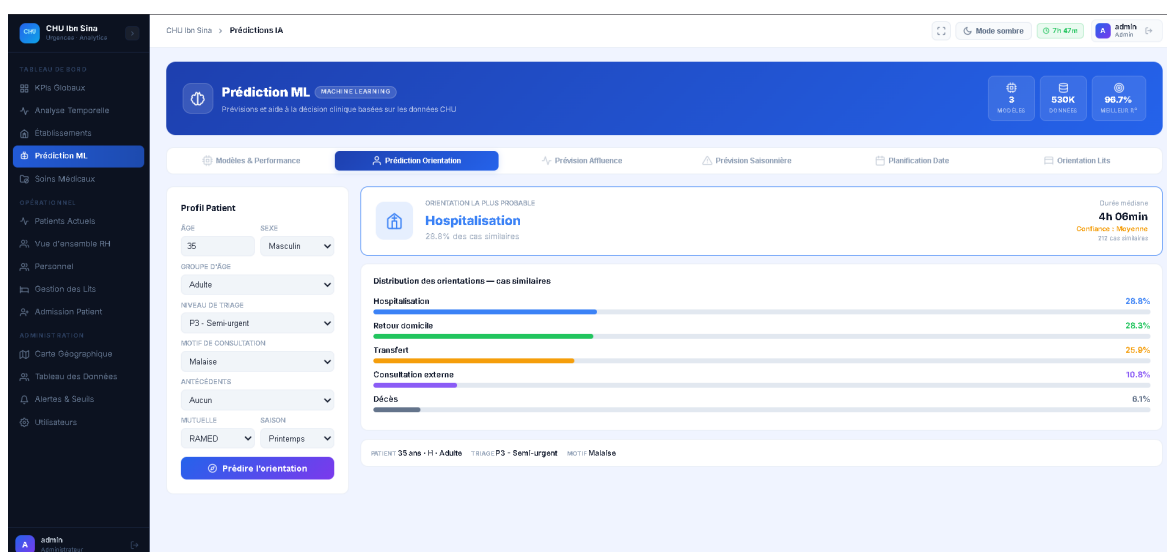
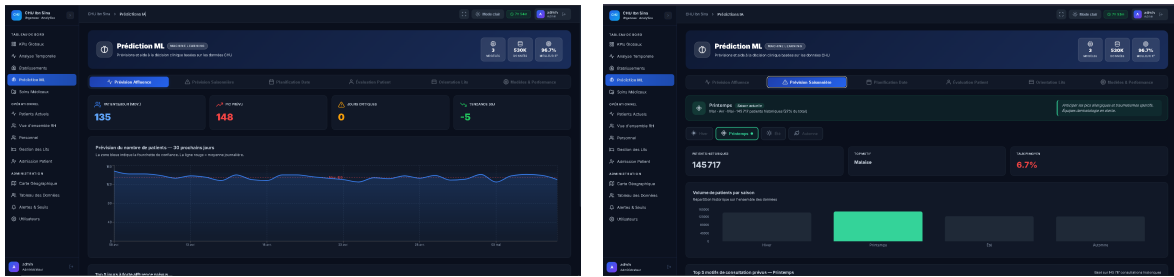


FIGURE 5.4 – Prédiction d'orientation – Distribution des orientations basée sur les cas historiques similaires

Prévision d'affluence et saisonnière

Le module de prévision utilise XGBoost pour anticiper le nombre de patients sur les 30 prochains jours, en tenant compte des facteurs saisonniers, des jours fériés et des tendances historiques. La prévision saisonnière identifie les pathologies dominantes selon la saison pour préparer les stocks médicamenteux et les affectations du personnel.



(a) Prédiction d'affluence 30 jours (XGBoost) (b) Prédiction saisonnière des pathologies

FIGURE 5.5 – Modules de prévision des flux de patients

Planification par date

Pour une date future donnée, ce module calcule le nombre de patients attendus, les ressources humaines recommandées par spécialité médicale et le nombre de lits nécessaires par service. Les recommandations sont basées sur la moyenne historique du même jour de la semaine et du même mois, multipliée par un facteur d'ajustement selon la charge prévisionnelle.

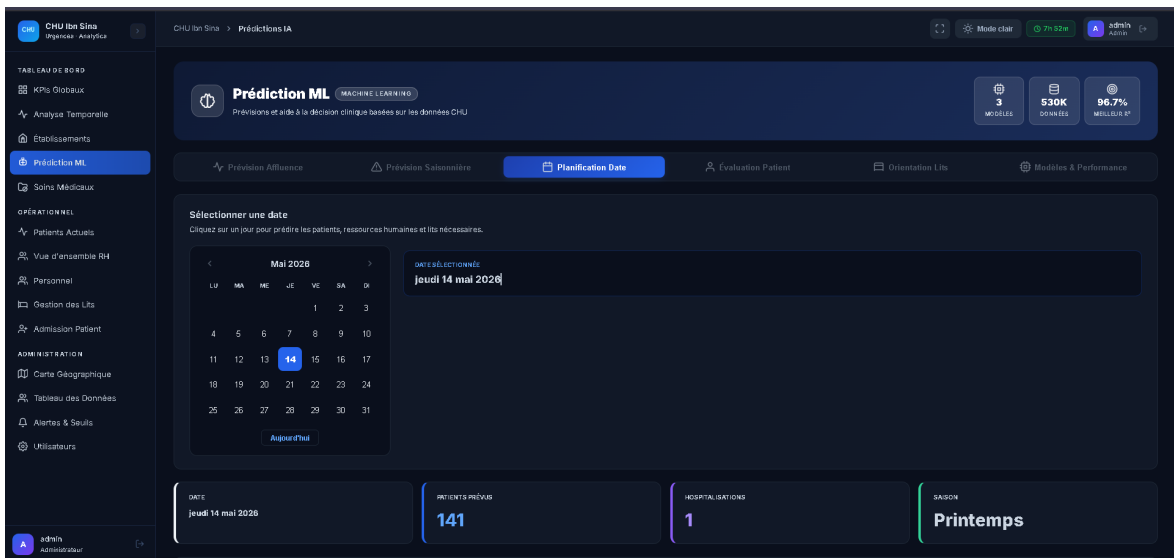


FIGURE 5.6 – Planification par date – Recommandations automatiques RH et lits

5.3.5 Page 5 – Ressources Humaines

La page Ressources Humaines offre une analyse croisée entre le flux de patients et les ressources disponibles. Elle permet aux responsables d'identifier en temps réel les déséquilibres entre la demande de soins et l'offre en personnel médical. Les graphiques de corrélation flux/médecins disponibles mettent en évidence les périodes de sous-effectif, tandis que les indicateurs de saturation par service alertent sur les situations critiques avant qu'elles ne se produisent.

Les indicateurs calculés comprennent le ratio médecins par patient, le taux d'occupation par spécialité, et la comparaison entre l'effectif actuel et l'effectif recommandé par les modèles de planification.

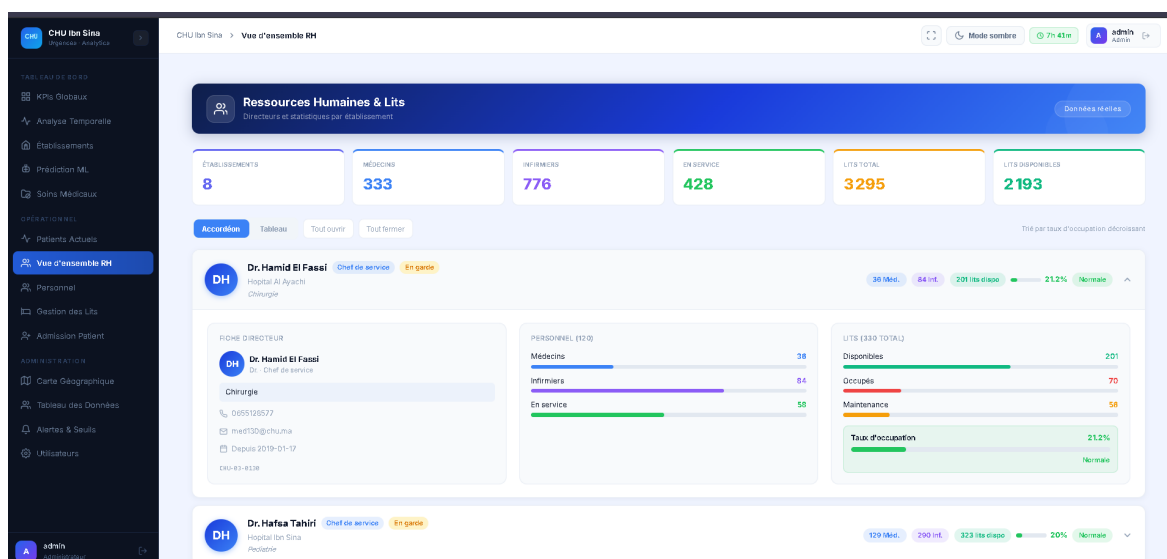


FIGURE 5.7 – Vue Ressources Humaines – Corrélation flux patients / personnel disponible par service

5.3.6 Page 6 – Soins Médicaux

Cette page analyse l'ensemble des 737 512 actes médicaux enregistrés dans la base de données. Elle présente la répartition des types de soins, les coûts moyens par établissement et par service, ainsi que la distribution des modes de financement (AMO, RAMED, Payant, Assurance, Exonération réglementée). Ces informations sont essentielles pour la direction financière afin d'optimiser la politique tarifaire et de piloter les conventions avec les organismes de remboursement.

Les graphiques permettent d'identifier les actes les plus coûteux, les services les plus consommateurs de ressources et les tendances d'évolution des coûts dans le temps.

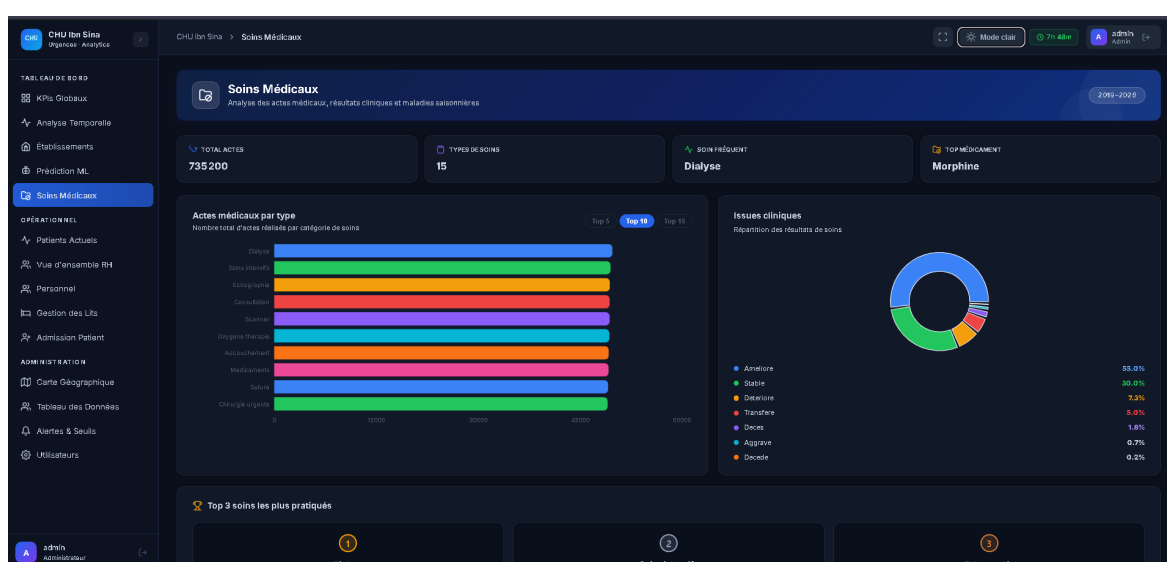


FIGURE 5.8 – Page Soins Médicaux – Analyse des coûts et répartition des financements

5.3.7 Page 7 – Patients Actuels

La page Patients Actuels constitue le module de suivi opérationnel en temps réel. Elle affiche l'ensemble des patients admis dans les **24 dernières heures glissantes** et se rafraîchit automatiquement toutes les 30 secondes. Chaque patient est affiché avec son heure d'arrivée, son niveau de triage (code couleur P1 à P4), son motif de consultation et son statut clinique actuel.

Les fonctionnalités principales de cette page sont :

- **Gestion des statuts** : progression En triage → En attente → En traitement → Sorti, modifiable en un clic ;
- **Attribution de lit** : sélection du lit disponible depuis un menu déroulant groupé par hôpital, avec mise à jour immédiate en base de données ;
- **Filtres combinables** : par statut, niveau de triage (P1 à P4) et recherche textuelle sur le nom, CIN ou motif ;
- **Double vue** : mode accordéon (regroupement par établissement) et mode tableau global ;
- **KPIs temps réel** : admissions du jour, patients actifs, taux d'occupation des lits, taux de criticité P1.

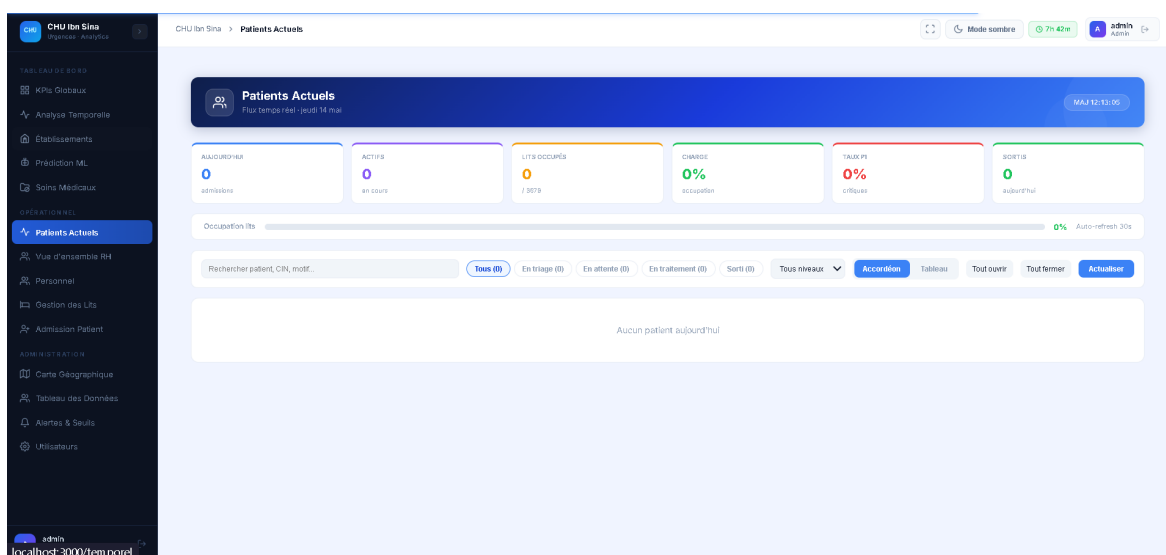


FIGURE 5.9 – Page Patients Actuels – Suivi temps réel avec KPIs et attribution de lits

5.3.8 Page 8 – Admission Patient

Le formulaire d'admission permet de saisir un nouveau patient directement depuis l'interface web, sans nécessiter d'accès au système d'information hospitalier sous-jacent. Il comprend l'ensemble des champs nécessaires à la constitution du dossier urgences : données d'identité, groupe sanguin, antécédents, niveau de triage, motif de consultation, orientation prévisionnelle et informations financières.

Plusieurs mécanismes automatiques facilitent la saisie :

- La **date et l'heure d'arrivée** sont pré-remplies à l'heure locale du serveur ;

- Le **calcul automatique des tarifs** s'adapte à la mutuelle sélectionnée ;
- La liste des **établissements** est chargée dynamiquement depuis la base de données ;
- Une **protection** empêche la saisie de dates futures ou antérieures de plus de 24h.

FIGURE 5.10 – Formulaire d'admission – Saisie complète d'un nouveau patient aux urgences

5.3.9 Page 9 – Gestion des Lits

La page Gestion des Lits offre une vue exhaustive des 3 295 lits répartis dans les 8 hôpitaux du réseau CHU Ibn Sina. Chaque lit est caractérisé par son numéro (selon la convention de nommage standardisée : UR pour Urgences, RE pour Réanimation, CH pour Chirurgie, etc.), son service, son type (Standard, Soins intensifs, Réanimation, Pédiatrique) et son statut (Disponible, Occupé, En maintenance).

Le tableau de bord de cette page affiche en temps réel le taux d'occupation global et par service, permettant aux équipes logistiques d'anticiper les besoins en lits et d'optimiser les rotations. La modification du statut d'un lit s'effectue directement depuis l'interface.

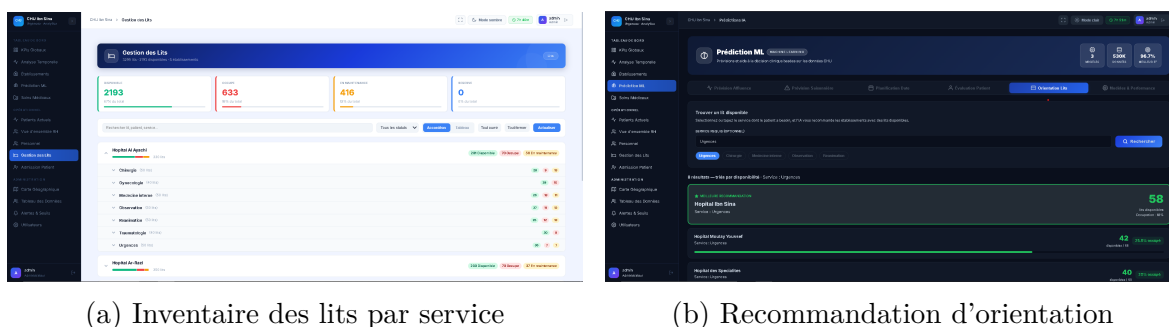


FIGURE 5.11 – Page Gestion des Lits – Inventaire et taux d'occupation en temps réel

5.3.10 Page 10 – Gestion du Personnel

La page Gestion du Personnel présente l'annuaire complet des 1 086 membres du personnel médical et soignant. Chaque fiche contient la spécialité médicale, le statut de service (En service / En garde / En congé), le matricule et l'établissement d'affectation. Des filtres par spécialité et par établissement permettent une navigation rapide dans l'annuaire. Un graphique de répartition visualise la distribution des spécialités au sein de chaque hôpital, facilitant le repérage des services sous-dotés.

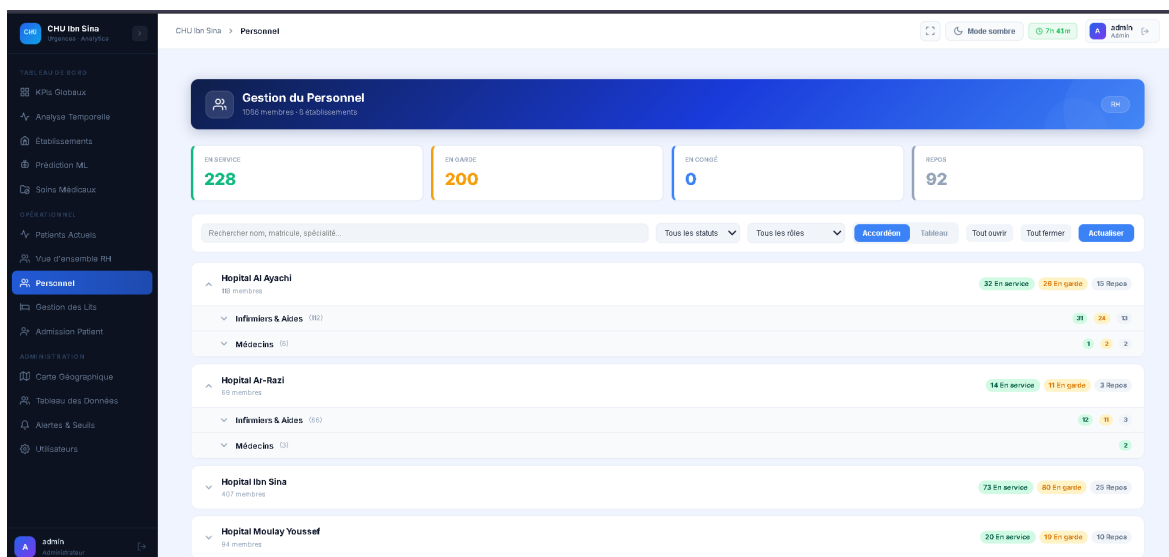
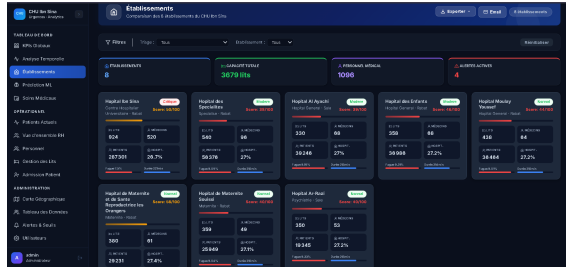
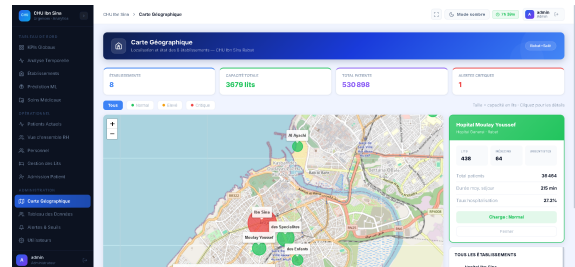


FIGURE 5.12 – Page Gestion du Personnel – Annuaire et répartition par spécialité

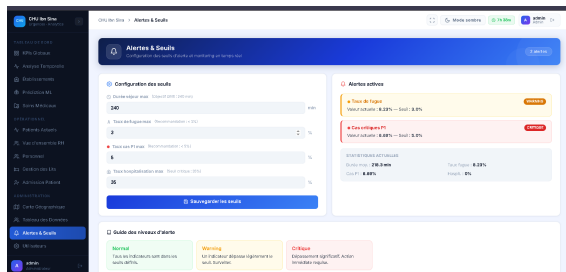
5.3.11 Pages 11 à 15



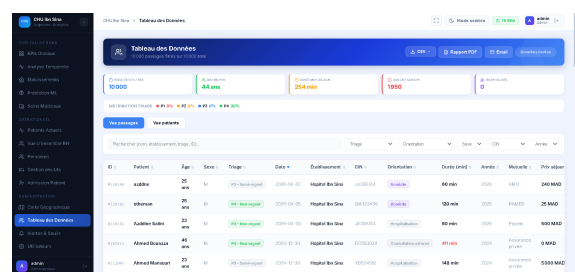
(a) Établissements (P.11)



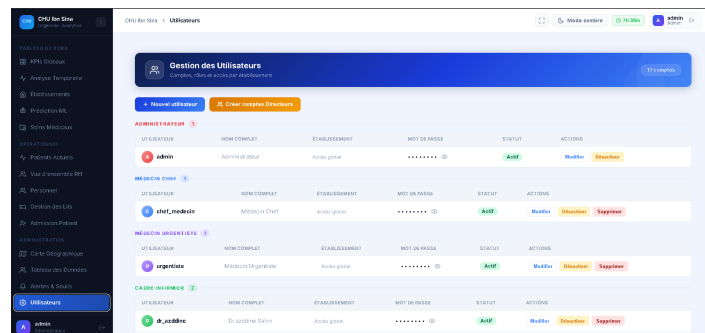
(b) Carte Géographique (P.12)



(c) Alertes & Seuils (P.13)



(d) Tableau des Données (P.14)



(e) Gestion Utilisateurs (P.15)

FIGURE 5.13 – Pages 11 à 15 du tableau de bord

5.4 Architecture de l'application

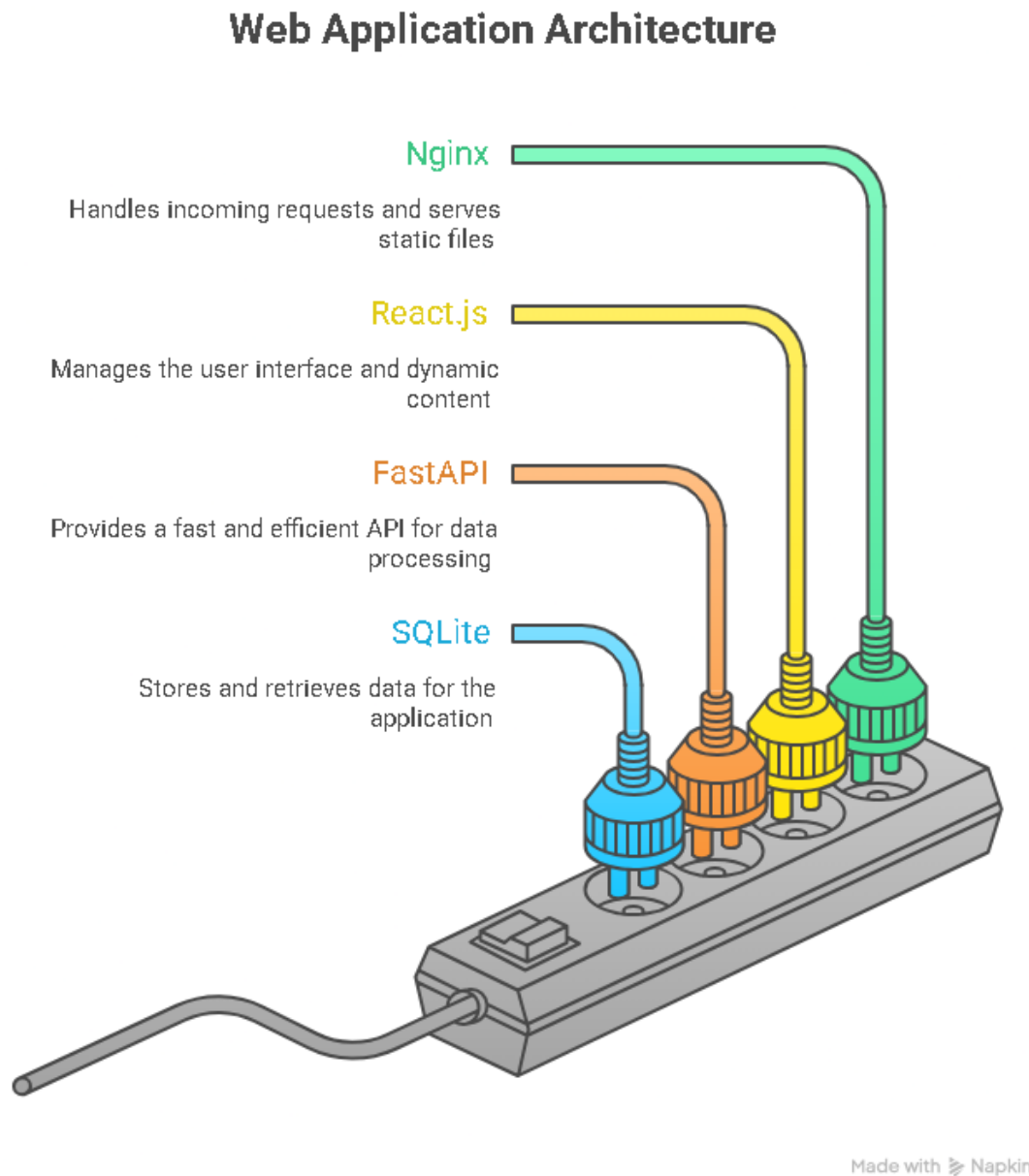


FIGURE 5.14 – Architecture générale – Docker + FastAPI + React + MySQL + Cache Parquet

5.5 Conclusion

Les 15 pages du tableau de bord couvrent l'intégralité du cycle opérationnel des urgences : analyse historique, prédiction, suivi temps réel et planification. L'architecture modulaire en 12 routers backend facilite la maintenance et les évolutions futures.

Chapitre 6

Sécurité, Performance et Déploiement

6.1 Introduction

Ce chapitre traite des aspects transversaux de l'application : la sécurité des accès, les performances du système et le déploiement via Docker Compose.

6.2 Sécurité de l'application

6.2.1 Authentification par JSON Web Token (JWT)

L'authentification est gérée via des tokens JWT (RFC 7519) qui encodent l'identité et les permissions de l'utilisateur :

Listing 6.1 – Generation et validation JWT

```
1 import jwt as pyjwt
2
3 JWT_SECRET = os.environ.get("JWT_SECRET", "changeme")
4 JWT_EXPIRE = int(os.environ.get("JWT_EXPIRE_MINUTES", 480))
5
6 def create_jwt(username: str, role: str, etablissement: str)
7     -> str:
8     payload = {
9         "sub": username,
10         "role": role,
11         "etab": etablissement,
```

```
11         "exp": datetime.utcnow() +
12             timedelta(minutes=JWT_EXPIRE),
13         "iat": datetime.utcnow(),
14     }
15     return pyjwt.encode(payload, JWT_SECRET, algorithm="HS256")
16
17 def verify_token(creds: HTTPAuthorizationCredentials) -> dict:
18     try:
19         return pyjwt.decode(creds.credentials, JWT_SECRET,
20                             algorithms=["HS256"])
21     except pyjwt.ExpiredSignatureError:
22         raise HTTPException(401, "Token expiré")
23     except pyjwt.InvalidTokenError:
24         raise HTTPException(401, "Token invalide")
```

6.2.2 Hachage des mots de passe

Les mots de passe sont hachés avec SHA-256 avant stockage :

Listing 6.2 – Hachage des mots de passe

```
1 import hashlib
2
3 def _hash(pwd: str) -> str:
4     return hashlib.sha256(pwd.encode()).hexdigest()
5
6 # Vérification lors de la connexion
7 if stored_hash != _hash(body.password):
8     raise HTTPException(401, "Identifiants incorrects")
```

6.2.3 Contrôle d'accès basé sur les rôles (RBAC)

Chaque endpoint vérifie le rôle de l'utilisateur avant d'exécuter la requête :

Listing 6.3 – Guard RBAC – acces admin uniquement

```
1 def _require_admin(payload: dict):
2     if payload.get("role") != "admin":
3         raise HTTPException(403, "Accès réservé aux
4             administrateurs")
5
6 @router.get("/api/users")
7 def get_users(creds = Depends(security)):
8     payload = verify_token(creds)
9     _require_admin(payload)          # Guard admin
10    # ... logique métier
```

6.2.4 Filtrage par établissement

Les directeurs ne voient que les données de leur établissement :

Listing 6.4 – Filtrage par etablissement (directeur)

```
1 def _etab_from_creds(creds) -> str | None:
2     """Retourne l'etablissement du directeur, ou None pour
3         admin/medecin."""
4     try:
5         payload = verify_token(creds)
6         if payload.get("role") == "directeur":
7             return payload.get("etab")
8     except Exception:
9         pass
10    return None
11
12 # Dans chaque endpoint
13 etab_user = _etab_from_creds(creds)
14 if etab_user:
15     df = df[df["Etablissement"] == etab_user]
```

6.3 Performance et optimisation

6.3.1 Cache Parquet

Le principal défi de performance était le temps de chargement des 530 914 lignes depuis MySQL à chaque démarrage. La solution adoptée repose sur un cache Parquet :

TABLE 6.1 – Comparaison des temps de chargement

Mode	Temps (urgences)	Temps (soins)
MySQL direct (sans cache)	~25–30 secondes	~45 secondes
Cache Parquet (warm)	~0.3 secondes	~0.5 secondes
Gain	×90	×90

6.3.2 Stratégie de mise en cache

1. Au démarrage, le backend vérifie l'existence du fichier `urgences.parquet` ;
2. Si présent et à jour (`COUNT(*) DB = COUNT cache`), chargement immédiat depuis Parquet ;
3. Si MySQL a plus de lignes (nouveau patient admis), rechargement depuis MySQL ;
4. Après chaque rechargement MySQL, le Parquet est mis à jour automatiquement.

6.3.3 Temps de réponse des API

TABLE 6.2 – Temps de réponse mesurés pour les principales API

Endpoint	Temps de réponse (cache chaud)
GET /api/kpis	< 50 ms
GET /api/urgences/temporel	< 120 ms
GET /api/patients/aujourd_hui	< 80 ms
POST /api/simulateur	< 200 ms
GET /api/lits	< 30 ms

6.4 Déploiement Docker

6.4.1 Configuration Docker Compose

Listing 6.5 – docker-compose.yml

```
1 services:
2   backend:
3     build: ./backend
4     container_name: chu_backend
5     ports:
6       - "8000:8000"
7     volumes:
8       - ./data:/app/data
9       - ./models:/app/models
10    environment:
11      -
12        DATABASE_URL=mysql+pymysql://root:@host.docker.internal:3306/chu_
13      - JWT_SECRET=${JWT_SECRET}
14      - JWT_EXPIRE_MINUTES=480
15    restart: unless-stopped
16
17  frontend:
18    build: ./frontend
19    container_name: chu_frontend
20    ports:
21      - "3000:80"
22    restart: unless-stopped
```

6.4.2 Commandes de déploiement

Listing 6.6 – Commandes de gestion des conteneurs

```
1 # Demarrage complet (backend + frontend)
2 docker compose up -d --build
3
4 # Reconstruire uniquement le backend
```

```
5 docker compose up -d --build backend
6
7 # Consulter les logs en temps reel
8 docker logs chu_backend -f --tail=50
9
10 # Etat des conteneurs
11 docker compose ps
```

6.4.3 URLs d'accès

TABLE 6.3 – URLs d'accès à l'application

Service	URL	Description
Frontend	http://localhost:3000	Interface utilisateur React
API REST	http://localhost:8000	Backend FastAPI
Swagger UI	http://localhost:8000/docs	Documentation interactive
phpMyAdmin	http://localhost/phpmyadmin	Interface MySQL (XAMPP)

6.5 Tests et validation

6.5.1 Stratégie de test

La validation de l'application a été conduite selon trois niveaux complémentaires :

1. **Tests fonctionnels** : vérification de chaque endpoint API via la documentation Swagger UI (/docs) — 64 endpoints testés manuellement ;
2. **Tests d'intégration** : validation de la chaîne complète admission → MySQL → cache → affichage frontend ;
3. **Tests de régression** : vérification qu'aucune page existante n'est cassée après chaque modification du backend.

6.5.2 Scénarios de test critiques

TABLE 6.4 – Scénarios de test et résultats

Scénario	Description	Attendu	Résultat
Admission patient	Saisie formulaire + vérification MySQL	IPP créé	OK
Patients Actuels 24h	Patient admis apparaît dans la liste	Visible <30s	OK
Attribution lit	Sélection lit + mise à jour statut	Lit assigné en DB	OK
Démarrage cache	Redémarrage conteneur avec cache Parquet	< 1 seconde	OK
Token expiré	Requête avec JWT périmé	Erreur 401	OK
Accès directeur	Directeur voit seulement son établissement	Données filtrées	OK
Pipeline ETL	Lancement pipeline en mode incrémental	Pas de doublons	OK
Date invalide	Admission avec date future	Clampage serveur	OK
MySQL hors ligne	Démarrage sans MySQL	Cache Parquet OK	OK
Prédiction planif.	Planification pour une date future	RH + lits affichés	OK

6.5.3 Validation avec les utilisateurs finaux

Une session de démonstration a été organisée avec les équipes du CHU Ibn Sina. Les retours recueillis ont permis d'identifier et de corriger plusieurs points :

- **Nommage des lits** : les anciens formats A-10, LIT-000011 ont été uniformisés en préfixes métier (UR-01 pour Urgences, RE-01 pour Réanimation, etc.) ;
- **Fenêtre 24h glissante** : le filtre initial "aujourd'hui depuis minuit" a été étendu aux 24 dernières heures glissantes pour ne pas perdre les patients admis en fin de nuit précédente ;
- **Timeout chargement** : un délai maximum de 20 secondes a été introduit pour éviter que la page de chargement ne bloque indéfiniment en cas d'indisponibilité MySQL.

6.6 Conclusion

L'application déployée répond aux exigences de sécurité (JWT + RBAC + SHA-256), de performance (cache Parquet, <200ms en réponse API) et de portabilité (Docker Compose). Les tests fonctionnels et la validation avec les utilisateurs finaux confirment que le système est prêt pour un déploiement en environnement hospitalier de production.

Conclusion Générale

Bilan du projet

Ce projet de fin d'études nous a permis de concevoir, développer et déployer une solution complète de *Business Intelligence* et de *Data Science* appliquée au domaine hospitalier. La solution couvre l'intégralité du cycle de vie de la donnée : de l'ingestion brute jusqu'à la visualisation décisionnelle et la prédiction.

Les principaux apports techniques de ce travail sont :

- Un **pipeline ETL Medallion** (Bronze/Silver/Gold) garantissant la qualité, la traçabilité et la reproductibilité des traitements sur 530 914 enregistrements ;
- Une **architecture full-stack découplée** (FastAPI + React TypeScript) avec 64 endpoints REST documentés, organisés en 12 routers thématiques ;
- Trois **modèles ML** opérationnels : XGBoost (prédiction flux), Random Forest (prédiction d'orientation patient), Prophet (prévision séries temporelles 30 jours) ;
- Un **système de cache Parquet** multipliant la vitesse de démarrage par 90 (de 30 secondes à moins d'une seconde) ;
- Un **déploiement Docker Compose** assurant la portabilité, la reproductibilité et la scalabilité de la solution.

Apports pour le CHU Ibn Sina

Sur le plan métier, l'application offre au personnel médical et administratif du CHU Ibn Sina :

- Une vision consolidée en temps réel de l'activité des urgences, actualisée toutes les 30 secondes ;

- La capacité d’anticiper les pics d’affluence jusqu’à 30 jours à l’avance, permettant un ajustement proactif du personnel et des ressources ;
- Un suivi individuel de chaque patient, de son admission à sa sortie, avec attribution automatisée des lits disponibles ;
- Un système d’alertes configurables sur les indicateurs critiques, avec code couleur intuitif (vert / orange / rouge).

Perspectives d’évolution

Plusieurs axes d’amélioration peuvent être envisagés pour les versions futures :

1. **Enrichissement des modèles ML** : intégration de données exogènes (météo, calendrier épidémique, jours fériés) pour améliorer la précision des prévisions ;
2. **Application mobile** : développement d’une version mobile (React Native ou PWA) pour les soignants en déplacement dans les services ;
3. **Extension des données** : extension du pipeline ETL aux données de consultation externe, d’hospitalisation et de laboratoire pour une vision 360° de l’activité ;
4. **Deep Learning** : remplacement de Prophet par un modèle LSTM pour capturer des dépendances temporelles plus complexes ;
5. **Alertes en temps réel** : envoi d’alertes par SMS et email via un service de messaging (Twilio, SendGrid) lorsqu’un seuil critique est dépassé ;
6. **Interopérabilité HL7/FHIR** : connecter l’application aux systèmes d’information hospitaliers standardisés (HIS) pour automatiser l’ingestion des données.

Retour d’expérience

Ce projet m’a permis de développer et d’approfondir des compétences transversales couvrant l’ensemble du spectre du Data Engineering : collecte et préparation des données, modélisation, développement backend et frontend, sécurisation des accès, optimisation des performances et déploiement en production. La confrontation avec les contraintes réelles du terrain hospitalier a été particulièrement enrichissante et formatrice.

Bibliographie

- [1] Ramírez, S. (2018). *FastAPI – Modern, Fast Web Framework for Python*. <https://fastapi.tiangolo.com>
- [2] Facebook Inc. (2013). *React – A JavaScript Library for Building User Interfaces*. <https://react.dev>
- [3] McKinney, W. (2010). *Data Structures for Statistical Computing in Python*. Proceedings of the 9th Python in Science Conference, 51–56.
- [4] Chen, T., & Guestrin, C. (2016). *XGBoost : A Scalable Tree Boosting System*. Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining, 785–794.
- [5] Breiman, L. (2001). *Random Forests*. Machine Learning, 45(1), 5–32.
- [6] Taylor, S. J., & Letham, B. (2018). *Forecasting at Scale*. The American Statistician, 72(1), 37–45.
- [7] Merkel, D. (2014). *Docker : Lightweight Linux Containers for Consistent Development and Deployment*. Linux Journal, 2014(239).
- [8] Databricks. (2021). *Medallion Architecture : A Best Practice for Building a Robust Data Lakehouse*. <https://www.databricks.com/glossary/medallion-architecture>
- [9] Oracle Corporation. (2023). *MySQL 8.0 Reference Manual*. <https://dev.mysql.com/doc/refman/8.0/en/>
- [10] Bayer, M. (2012). *SQLAlchemy – The Database Toolkit for Python*. <https://www.sqlalchemy.org>
- [11] Jones, M., Bradley, J., & Sakimura, N. (2015). *JSON Web Token (JWT)*. RFC 7519, Internet Engineering Task Force (IETF).
- [12] Apache Software Foundation. (2013). *Apache Parquet – Columnar Storage Format*. <https://parquet.apache.org>
- [13] Haux, R. (2006). *Health Information Systems – Past, Present, Future*. International Journal of Medical Informatics, 75(3–4), 268–281.

Annexe A

Annexes

A.1 Structure complète du projet

Listing A.1 – Arborescence complète du projet PFE

```
1 PFE/
2 +-- backend/
3 |   +-- main.py           # FastAPI : app, middleware,
   startup
4 |   +-- etl_pipeline.py   # Pipeline ETL standalone
5 |   +-- core/
6 |   |   +-- database.py   # Engine SQLAlchemy, data dict
7 |   |   +-- cache.py      # Cache Parquet (save/load)
8 |   |   +-- data_loader.py # get_urg(), reload_urg()
9 |   |   +-- auth.py       # JWT, hachage, roles RBAC
10 |   +-- routers/         # 12 routers -- un par
   fonctionnalite
11 |       +-- auth.py      # Login, logout, me, refresh
12 |       +-- dashboard.py # KPIs, stats, status
13 |       +-- urgences.py  # Analyse temporelle, heatmap
14 |       +-- patients.py  # Patients actuels, admission
15 |       +-- soins.py     # Soins medicaux
16 |       +-- ressources.py # Personnel + Lits
17 |       +-- etablisements.py # Etablissements + carte
18 |       +-- predictions.py # ML + simulateur +
   planification
19 |       +-- alertes.py    # Alertes configurables
```

```

20 |      +-- export.py           # Export Excel/CSV + email
21 |      +-- pipeline.py        # ETL Bronze->Gold
22 |      +-- users.py           # Gestion utilisateurs CRUD
23 +-- frontend/
24 |   +-- src/
25 |       +-- pages/           # 15 pages React
26 |       +-- components/      # 17 composants reutilisables
27 |       +-- context/         # Theme, Auth, Toast
28 |       +-- hooks/           # useAutoRefresh, usePageTheme
29 +-- data/
30 |   +-- bronze/              # urgences_bronze.csv
    |   (immutable)
31 |   +-- silver/              # urgences_silver.csv (genere)
32 |   +-- gold/                # urgences_gold_processed.csv
33 |   +-- cache/               # urgences.parquet,
    |   soins.parquet
34 |   +-- logs/                # etl.log
35 +-- models/
36 |   +-- xgboost_model.pkl
37 |   +-- random_forest_model.pkl
38 |   +-- label_encoder.pkl
39 |   +-- metrics_comparison.csv
40 +-- docker-compose.yml
41 +-- .env                     # DATABASE_URL, JWT_SECRET,
    SMTP

```

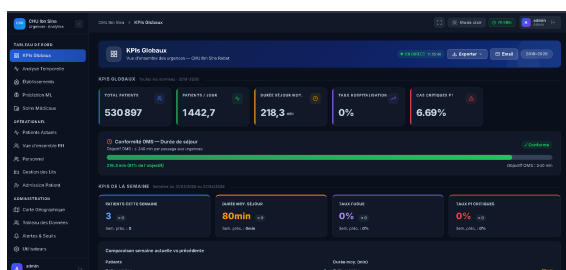
A.2 Endpoints API complets

Méthode	Route	Description
POST	/api/auth/login	Authentification JWT
POST	/api/auth/logout	Révocation session
GET	/api/auth/me	Profil utilisateur
POST	/api/auth/refresh	Renouvellement token
GET	/api/status	État du serveur et des données

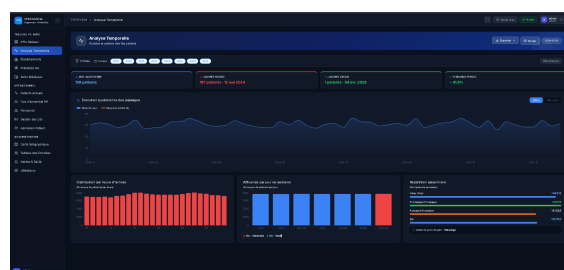
Méthode	Route	Description
GET	/api/kpis	KPIs globaux filtrés
GET	/api/kpis/live	KPIs temps réel
GET	/api/stats/resume	Résumé statistique
GET	/api/stats/comparaison	Comparaison inter-établissements
GET	/api/urgences/temporel	Série temporelle quotidienne
GET	/api/urgences/horaire	Distribution horaire
GET	/api/urgences/triage	Distribution par triage
GET	/api/urgences/orientation	Distribution par orientation
GET	/api/urgences/annuel	Évolution annuelle
GET	/api/urgences/saison	Distribution saisonnière
GET	/api/urgences/top_motifs	Top motifs de consultation
GET	/api/urgences/maladies_saison	Maladies saisonnières
GET	/api/urgences/liste	Liste paginée avec filtres
GET	/api/heatmap	Heatmap heure $\times$ jour
POST	/api/patients/add	Admission nouveau patient
GET	/api/patients/aujourd_hui	Patients dernières 24h
PATCH	/api/patients/IPP/statut	Mise à jour statut + lit
GET	/api/soins/types	Types d'actes médicaux
GET	/api/soins/couts_par_type	Coûts par type de soin
GET	/api/soins/couts_par_etab	Coûts par établissement
GET	/api/ressources	Vue RH globale
GET	/api/personnel	Liste du personnel
PATCH	/api/personnel/matricule	Mise à jour statut personnel
GET	/api/lits	Inventaire complet des lits
GET	/api/lits/stats	Statistiques d'occupation
GET	/api/lits/disponibles	Lits disponibles groupés par hôpital
PATCH	/api/lits/etab/num	Mise à jour statut lit
GET	/api/etablissements	Liste des établissements
GET	/api/etablissements/carte	Données géolocalisées
GET	/api/predictions	Prévisions Prophet 30 jours
POST	/api/simulateur	Prédiction XGBoost

Méthode	Route	Description
POST	/api/predict/orientation	Prédiction RF de l'orientation patient
GET	/api/predictions/planification	Planification RH et lits par date
GET	/api/alertes	État des alertes actives
GET	/api/alertes/check	Vérification des seuils
GET	/api/alertes/config	Lire la configuration
POST	/api/alertes/config	Modifier les seuils
GET	/api/export/urgences	Export CSV urgences
GET	/api/export/soins	Export CSV soins
GET	/api/export/excel	Export Excel complet
POST	/api/email/send	Envoi de rapport par email
GET	/api/pipeline/status	État des couches Bronze/Gold
POST	/api/pipeline/run	Lancer le pipeline ETL
GET	/api/users	Liste des utilisateurs
POST	/api/users	Créer un utilisateur
PATCH	/api/users/username	Modifier un utilisateur
DELETE	/api/users/username	Supprimer un utilisateur

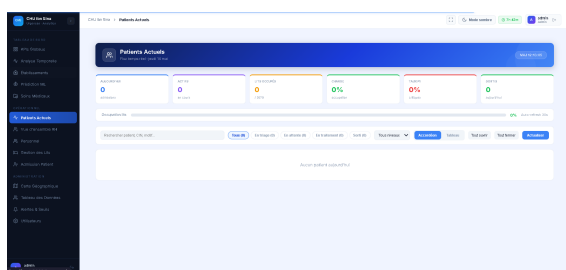
A.3 Galerie des captures d'écran



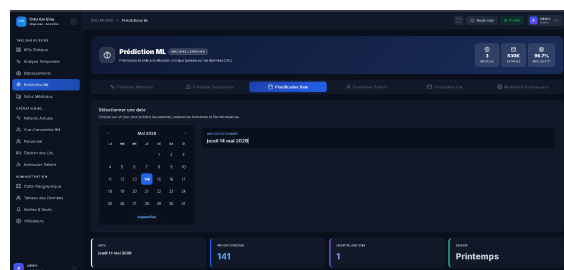
(a) KPIs Globaux



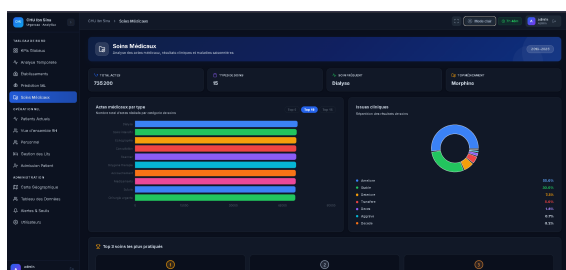
(b) Analyse Temporelle



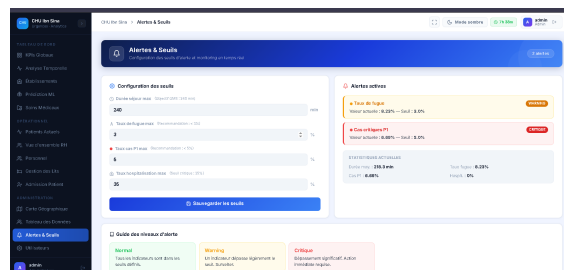
(c) Patients Actuels



(d) Planification par Date



(e) Soins Médicaux



(f) Alertes & Seuils

FIGURE A.1 – Galerie des pages du tableau de bord CHU Ibn Sina